Sapienza
*Università di Roma*

Ingegneria dell'Informazione, Informatica e Statistica

Laurea Triennale

Dipartimento di Informatica

# Algoritmi II

Appunti a cura di
**Lorenzo Ferraro**

12 novembre 2026

© 2026

Distribuito sotto licenza CC BY-NC-SA 4.0

https://creativecommons.org/licenses/by-nc-sa/4.0/deed.it

# Informazioni e Segnalazioni

Questi appunti raccolgono i contenuti del corso di *Algoritmi II*, tenuto nell'ambito del corso di laurea in Informatica presso "La Sapienza".

Il materiale viene mantenuto e aggiornato periodicamente sul mio repository personale `github.com/TonnoArcobaleno/uni-notes`, dove è possibile consultare la versione più recente e proporre correzioni o integrazioni.

Questi appunti non sostituiscono in alcun modo i libri di testo consigliati dai docenti o la partecipazione alle lezioni in aula, ma nascono con l'obiettivo di essere un supporto e agevolare lo studio e ripasso.

**Segnalazioni**

Trattandosi di appunti scritti e rivisti da uno studente, è possibile che contengano imprecisioni, refusi o passaggi poco chiari. Se ne individui uno, la segnalazione è benvenuta e può avvenire in due modi:

- aprendo una **Issue** sul repository indicato sopra, indicando il capitolo o la sezione interessata e descrivendo brevemente il problema riscontrato;
- scrivendo direttamente a **ferraro.notes.issues@proton.me**, specificando nell'oggetto il documento a cui ci si riferisce e, nel testo, il punto esatto e la natura dell'errore (è il canale preferito, perché più facile che io mi accorga della segnalazione in breve tempo).

Poiché il documento viene revisionato periodicamente, prima di segnalare un problema conviene verificare che non sia già stato corretto nell'ultima versione disponibile sul repository.

# Indice

**1 Grafi e Algoritmi sui grafi** **1**
1.1 Introduzione ai grafi . . . 1
1.2 Problema dei ponti di Konigsberg . . . 2
1.3 Rappresentare i grafi . . . 3
1.4 Cicli . . . 5
1.4.1 Problema sui cicli . . . 6
1.5 Percorrere un grafo . . . 8
1.6 DFS . . . 8
1.6.1 DFS nei grafi non connessi . . . 11
1.6.2 DFS Ricorsivo . . . 12
1.6.3 Alberi di ricerca . . . 12
1.6.4 Problema degli esami . . . 13
1.6.5 DFS per i grafi diretti . . . 15
1.6.6 Trovare un ordine topologico in un grafo diretto aciclico . . . 17
1.6.7 DFS impossibili . . . 23
1.6.8 Pozzi universali . . . 24
1.6.9 Ponti . . . 25
1.6.10 Grafi fortemente connessi . . . 28
1.7 BFS . . . 39
1.7.1 Trovare quanti cammini minimi esistono tra due nodi . . . 41
1.7.2 Distanza tra due sottoinsiemi . . . 43
1.7.3 Alberi BFS . . . 44

# Capitolo 1

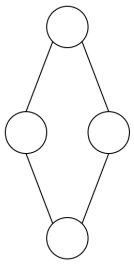
# Grafi e Algoritmi sui grafi

## 1.1 Introduzione ai grafi

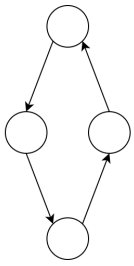
**Definizione 1.1 – Grafo**

Un **grafo** è una coppia ordinata $G = (V, E)$ dove $V$ è l'insieme dei **nodi** (o vertici), mentre $E$ è l'insieme degli **archi** che collegano due vertici.

In questi appunti guarderemo a grafi senza loop (ossia senza archi che collegano un nodo a sé stesso) e senza archi paralleli (ossia grafi che non hanno due o più archi uguali).



Questo è l'esempio più classico di grafo: i nodi sono i pallini, che sono collegati da delle linee che rappresentano gli archi.



Gli archi possono avere anche un orientamento, e se tutti gli archi lo hanno diciamo che il grafo è **orientato**, come il grafo qui a sinistra. In questo caso è possibile avere due archi differenti tra due nodi $x$ e $y$, ma devono avere la freccia invertita (ossia uno deve portare da $x$ a $y$ e l'altro da $y$ a $x$).

**Proprietà 1.1 – Archi**

Preso un arco $f$ da $x$ a $y$, diciamo che

- $f$ è **incidente** a $x$ e $y$
- $x$ è **adiacente** a $y$ ($x \sim y$)

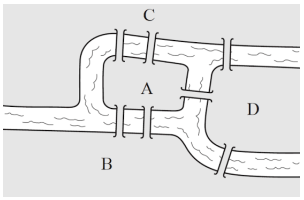
**Definizione 1.2 – Grado di un vertice**

Il **grado di un vertice** $x$ (**deg**$(x)$) è il **numero di archi incidenti** ad esso.

Nei due esempi visti prima, il grado di ogni vertice era 2.

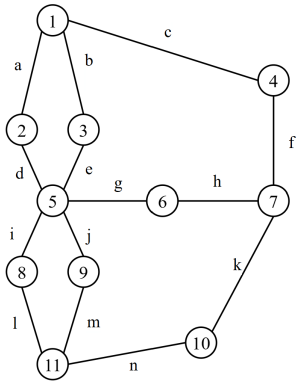
## 1.2 Problema dei ponti di Konigsberg

Il problema dei ponti di Konigsberg è uno dei problemi logico-matematici più famosi.




La domanda è semplice: con la seguente disposizione di ponti, è possibile attraversarli tutti una volta sola e tornare al punto di partenza?

Questo classico quesito è risolvibile usando i grafi. Infatti, possiamo schematizzarlo in questo modo:

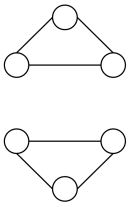



**Definizione 1.3 – Passeggiata**

Una **passeggiata** in un grafo è una **sequenza** del tipo $v_0e_1v_1e_2 \dots v_{n-1}e_nv_n$, dove $v_i$ e $v_{i+1}$ indicano due vertici collegati dall'arco $e_{i+1}$.
Una passeggiata è detta **chiusa** se $v_0 = v_n$.

Una passeggiata possibile nel grafo qui sopra ad esempio è "1 a 2 d 5 i 8 l 11 n 10 k 7".

Per risolvere il problema dei ponti di Konigsberg bisogna saper rispondere alla domanda "Quand'è che esiste una passeggiata chiusa in un grafo che contiene tutti gli archi presenti una volta sola?". La risposta è che **qualsiasi grafo con almeno un nodo $x$ tale che deg$(x)$%2 = 1** (ossia il cui grado è dispari) **non ammette una passeggiata di questo tipo**. Nel nostro caso, quindi, il problema è irrisolvibile: infatti i nodi 1, 5, 7 e 11 hanno un grado dispari. Ma se fossero tutti pari? Non è comunque detto.



In questo caso, infatti, nonostante tutti i nodi siano di grado 2 quindi pari, la risposta al problema è comunque no. È infatti comunque necessario che il grafo sia **connesso**.

**Definizione 1.4 – Grafo connesso**

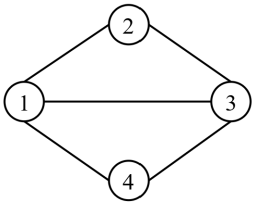
Un grafo si dice **connesso** se per ogni coppia di vertici $x$ e $y$ esiste una passeggiata che porti da $x$ a $y$.

**Proposizione 1.1**

Il problema dei ponti di Konigsberg è **risolvibile** $\iff$ **ogni vertice ha grado pari** e il grafo è **connesso**.

## 1.3 Rappresentare i grafi

Vedremo ora due modi differenti per rappresentare questo grafo:




**Definizione 1.5 – Matrice di adiacenze**

Si prende una matrice $M$ di dimensione $n \times n$ (dove $n = \#V$) tale che

$$m_{ij} = \begin{cases} \mathbf{1} & \textbf{se } x \sim y \\ \mathbf{0} & \textbf{altrimenti} \end{cases}$$

Nel nostro caso quindi abbiamo una matrice del tipo

$$M = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix} \end{array}$$

Questa matrice, dato che non abbiamo loop nei grafi, avrà sempre la diagonale principale composta solo da 0.

**Proposizione 1.2**

Se il grafo non è diretto, la matrice di adiacenze sarà **speculare**.

**Proprietà 1.2 – Costi computazionali matrice di adiacenze**

- **Memorizzazione del grafo**: $\Theta(n^2)$ (spaziale)
- **Visita del grafo**: $\Theta(n^2)$
- **Verificare se $i \sim j$**: $\Theta(1)$
- **Preso un nodo $x$, calcolare $\deg(x)$**: $\Theta(n)$

**Definizione 1.6 – Lista di adiacenza**

Si prende un array di dimensione n, e a ogni posizione i dell'array si punta ad una **lista puntata con tutti i vicini del nodo $i$-esimo**.

Nel nostro caso abbiamo quindi uno schema del tipo

$$\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} \begin{matrix} \longrightarrow [2,3,4] \\ \longrightarrow [1,3] \\ \longrightarrow [1,2,4] \\ \longrightarrow [1,3] \end{matrix}$$

**Proprietà 1.3 – Costi computazionali lista di adiacenze**

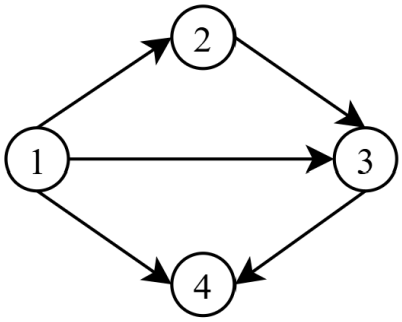
- **Memorizzazione del grafo**: $\Theta(n+m)$ (spaziale), dove $m = \#E$
- **Visita del grafo**: $\Theta(n+m)$
- **Verificare se $i \sim j$**: $\mathbf{O(\min(\deg(i), \deg(j)))}$
- **Preso un nodo $x$, calcolare $\deg(x)$**: $\mathbf{O}(\deg(x))$

In realtà, per la memorizzazione e la visita avremmo un costo pari a $\Theta(n + \sum_{v \in V} \deg(v))$, ma dato che $\Theta(n + \sum_{v \in V} \deg(v)) = \Theta(n + 2m) = \Theta(n + m)$, i conti tornano perfettamente.

Per i grafi diretti, invece, con le matrici di adiacenze scriviamo sulle colonne gli archi entranti in un nodo e sulle righe quelli uscenti, mentre con le liste di adiacenze ci sono 3 possibilità:

1) Le liste di adiacenze hanno solo i nodi in entrata
2) Le liste di adiacenze hanno solo i nodi in uscita
3) Ci sono due liste di adiacenze, una per i nodi in entrata e una per quelli in uscita

Tutti e 3 i modi sono perfettamente funzionanti. Vediamo con un esempio perché.




Se usiamo le liste di adiacenza con solo i valori in entrata, allora avremo uno schema del tipo

$$\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} \begin{matrix} \longrightarrow [] \\ \longrightarrow [1] \\ \longrightarrow [1,2] \\ \longrightarrow [1,3] \end{matrix}$$

mentre quelle con solo valori in output saranno del tipo

$$\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} \begin{matrix} \longrightarrow [2,3,4] \\ \longrightarrow [3] \\ \longrightarrow [4] \\ \longrightarrow [] \end{matrix}$$

In entrambi i casi, però, non stiamo perdendo informazioni sul grafo: infatti è possibile da queste liste ricreare il grafo da cui vengono senza alcun problema.

## 1.4 Cicli

**Definizione 1.7 – Cammino**

Un **cammino** è una passeggiata che **non ripete i vertici**.

**Proposizione 1.3**

Presi due nodi $x$ e $y$, **esiste un cammino** da $x$ a $y$ $\iff$ **esiste una passeggiata** da $x$ a $y$.

*Dimostrazione*

1° Esiste un cammino da $x$ a $y \implies$ esiste una passeggiata da $x$ a $y$
È ovvio, dato che un cammino è una passeggiata. ■

2° Esiste un cammino da $x$ a $y \impliedby$ esiste una passeggiata da $x$ a $y$
Prendiamo la passeggiata di lunghezza minima tra $x$ e $y$, e ipotizziamo per assurdo che non sia un cammino. Dunque sicuramente esiste un nodo che viene ripetuto, ossia esistono due indici $i < j$ tali che $v_i = v_j$. Possiamo allora tagliare il percorso che porta da $v_i$ di nuovo a sé stesso, dato che stiamo inutilmente allungando il giro. Ma così staremmo accorciando la passeggiata, che abbiamo detto essere di lunghezza minima. Abbiamo quindi trovato un assurdo, dunque abbiamo trovato che la passeggiata più corta è un cammino da $x$ a $y$. ■

**Definizione 1.8 – Ciclo**

Un **ciclo** è un grafo connesso dove ogni vertice è di grado 2.

### 1.4.1 Problema sui cicli

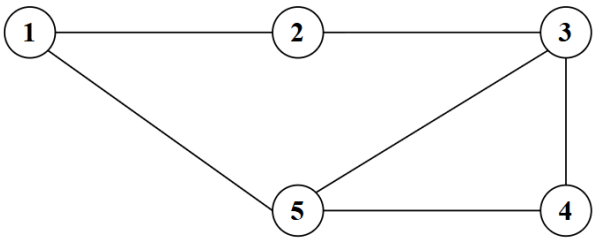
"Dato un grafo dove ogni nodo $x$ ha $\deg(x) \geq 2$, trovare un ciclo". Proviamo a studiare un algoritmo per risolvere questo problema.

Scegliamo un nodo di partenza $x$. Camminiamo lungo il grafo spostandoci sempre su un vicino diverso da quello dal quale siamo arrivati (il che è possibile dato che $\deg(x) \geq 2$). Prima o poi troveremo un vicino già visitato. In quel momento il nodo su cui ci troveremo avrà due vicini già visitati, il che significa che siamo arrivati ad un ciclo.

```
def trovaCiclo(G):
    L = lista vuota
    x = nodo qualsiasi
    L.append(x)
    current = x
    next = x.neighbors[0]
    while(next not in L):
        L.append(next)
        if next.neighbors[0] == current:
            u = next.neighbors[1]
        else:
            u = next.neighbors[0]
        current = next
        next = u
    while L[0] != next:
        L.remove(L[0])
    return L
```

L'ultimo while è fondamentale perché rimuove tutti i nodi che non fanno parte del ciclo.

Vediamo con un esempio grafico perché questo algoritmo funziona, studiando il grafo




Iniziamo con `L = [2]`, `current = 2` e `next = 3`. Dopo il primo ciclo avremo `L = [2,3]`, `current = 3` e `next = 4`. Dopo il secondo ciclo avremo `L = [2,3,4]`, `current = 4` e `next = 5`. Dopo il terzo ciclo avremo `L = [2,3,4,5]`, `current = 5` e `next = 3`. Dato che ora `next` è in `L`, usciamo dal ciclo. Nel secondo while eliminiamo 2, e dunque rimane `L = [3,4,5]`, che è proprio il ciclo nel grafo.

Questa versione, però, non è la più efficiente. Infatti, controllare che un elemento non sia in una lista non ordinata ha un costo $O(n)$, dunque, dato che il primo while è eseguito n volte, arriviamo ad un costo in $O(n^2)$. Vediamo un'altra versione più efficiente.

```
def trovaCicloEfficiente(G):
    n = numero di nodi di G
    L = lista vuota
    Vis = [0] * n
    x = nodo qualsiasi
    L.append(x)
    Vis[x] = 1
    current = x
    next = x.neighbors[0]
    while Vis[next] == 0:
        L.append(next)
        Vis[next] = 1
        if next.neighbors[0] == current:
            u = next.neighbors[1]
        else:
            u = next.neighbors[0]
        current = next
        next = u
    while L[0] != next:
        L.remove(L[0])
    return L
```

In questo caso usiamo quindi un array di supporto dove controlliamo se un nodo è già stato visitato o meno, riducendo il controllo della condizione del while da O($n$) a O(1), abbassando quindi il costo temporale a $\mathbf{O(n)}$. Il costo spaziale invece rimane identico in entrambi i casi: $\mathbf{O(n)}$.

## 1.5 Percorrere un grafo

Per percorrere un grafo partendo da un nodo $x$ e arrivando a un nodo $y$ abbiamo due possibili algoritmi:

- **DFS** (depth-first-search): partendo dal nodo $x$, passiamo ad uno dei suoi vicini non visitati fino a trovare un nodo i cui vicini sono tutti già visitati. A questo punto torniamo indietro fino ad un nodo che ha ancora vicini non visitati e continuiamo la ricerca fino a trovare il nodo $y$.
- **BFS** (breadth-first-search): partendo dal nodo $x$, controlliamo tutti i suoi vicini possibili segnandoli come visitati, e ripetiamo la stessa cosa per ogni vicino fino a trovare il nodo $y$.

## 1.6 DFS

Studiamo per primo l'algoritmo DFS.

Abbiamo quindi bisogno di un modo per controllare se un nodo sia stato controllato o meno in tempo costante. Possiamo usare un array `Vis`, di lunghezza n, dove

$$Vis[v_i] = \begin{cases} 1 & \text{se } v_i \text{ è stato visitato} \\ 0 & \text{se } v_i \text{ non è stato visitato} \end{cases}$$

Per salvare invece il nostro cammino momentaneo usiamo una **pila** (o stack). Questa ci permette di poter aggiungere ad ogni passo il nuovo vertice che stiamo studiando e ci permette di ripercorrere al contrario questo cammino quando siamo arrivati ad un nodo con vicini tutti già visitati. Vediamo quindi un primo algoritmo per la DFS:

```
DFS(G, x):
    n = numero di nodi di G
    Vis = [0] * n
    S = stack nuovo
    S.push(x)
    Vis[x] = 1
    while S != 0:
        y = S.top()
        if esiste z vicino di y tale che Vis[z] == 0:
            S.push(z)
            Vis[z] = 1
```

```
        else:
            S.pop()
    return Vis
```

**N.B.** Negli pseudocodici che vedremo da qui in avanti utilizziamo la convenzione con indici da 0 a $n-1$. Negli esempi pratici, tuttavia, i nodi dei grafi sono spesso numerati da 1 a $n$. Per applicare correttamente lo pseudocodice agli esempi sarebbe quindi necessario sottrarre 1 da ciascun nodo, ma per evitare di appesantire i codici noi qui omettiamo questa conversione, lasciando i nodi al loro valore standard.

**Proposizione 1.4**

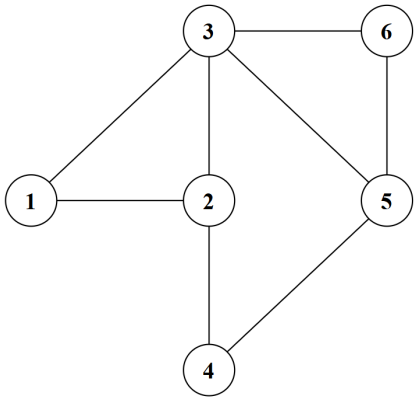
Presi due nodi $x$ e $y$ in un grafo, $y$ è raggiungibile da $x \implies$ `Vis[y] = 1`

*Dimostrazione*

Se $y$ è raggiungibile da $x$ allora sappiamo già che esiste un cammino $xe_1v_1 \dots v_{k-1}e_ky$, ossia un cammino da $x$ a $y$. Poniamo ora per assurdo che, dopo aver eseguito il nostro algoritmo di DFS, abbiamo `Vis[y] = 0`. Questo significa che esiste un indice i tale che `Vis[`$v_i$`] = 1` e `Vis[`$v_{i+1}$`] = 0`. Questo significa che $v_i$ è stato visitato, ma dato che l'algoritmo finisce solo quando `S == 0`, significa che $v_i$ è stato tolto dallo stack, ma questo si verifica solo quando tutti i vicini di $v_i$ sono stati visitati. Dunque anche $v_{i+1}$ è stato visitato, e dunque `Vis[`$v_{i+1}$`] = 1`. Abbiamo quindi trovato un assurdo, dunque nel nostro cammino non esiste un indice tale che `Vis[`$v_i$`] = 1` e `Vis[`$v_{i+1}$`] = 0`, dunque `Vis[y] = 1`. ■

Questo algoritmo quindi ritorna un array `Vis` in cui `Vis[y] = 1` $\iff$ $y$ è raggiungibile da $x$. Da destra verso sinistra è proprio la Proposizione 1.4; viceversa, da sinistra verso destra, vale perché un nodo viene segnato solo quando è vicino di un nodo già segnato, e quindi, per induzione, è raggiungibile da $x$.

Vediamo come funziona il nostro algoritmo con il seguente grafo:



Studiamo come ad ogni iterazione del ciclo while si modificano la pila S e l'array Vis.

1° `S = {1}` e `Vis = [1,0,0,0,0,0]`

2° `S = {1,2}` e `Vis = [1,1,0,0,0,0]`

3° `S = {1,2,3}` e `Vis = [1,1,1,0,0,0]`

4° `S = {1,2,3,5}` e `Vis = [1,1,1,0,1,0]`

5° `S = {1,2,3,5,6}` e `Vis = [1,1,1,0,1,1]`

6° `S = {1,2,3,5}` e `Vis = [1,1,1,0,1,1]`

7° `S = {1,2,3,5,4}` e `Vis = [1,1,1,1,1,1]`

8° `S = {1,2,3,5}` e `Vis = [1,1,1,1,1,1]`

9° `S = {1,2,3}` e `Vis = [1,1,1,1,1,1]`

10° `S = {1,2}` e `Vis = [1,1,1,1,1,1]`

11° `S = {1}` e `Vis = [1,1,1,1,1,1]`

12° `S = {}` e `Vis = [1,1,1,1,1,1]` $\implies$ Ritorniamo l'array `Vis = [1,1,1,1,1,1]`

Questo algoritmo ha un costo computazionale pari a $\mathbf{O}(n^2)$. Infatti, il while esegue un totale di $2n-1$ iterazioni, dato che nella pila S vengono inseriti $n-1$ nodi e ne vengono poi eliminati $n$, e dato che per ogni iterazione abbiamo un costo pari a $O(\deg(y))$ (dato che dobbiamo cercare un vicino $z$ tale che `Vis[z] == 0`, operazione che può richiedere nel caso peggiore $O(\deg(y))$), dove $y$ è il nodo in cima alla pila in quella iterazione, che arriva nel caso peggiore a valere $O(n)$, troviamo che $T(n) = O(n^2)$.

Possiamo però migliorare questa complessità temporale tenendo i vicini di ogni nodo in una lista puntata. Vediamo quindi un algoritmo alternativo per la DFS:

```
DFS_Alternativo(G,x):
    n = numero di nodi di G
    Vis = [0] * n
    S = stack nuovo
    S.push(x)
    Vis[x] = 1
    while S != 0:
        y = S.top()
        while y.adiacenti != 0:
            z = y.adiacenti[0]
            y.adiacenti.remove(z)
            if Vis[z] == 0:
                Vis[z] = 1
                S.push(z)
                break
        if S.top() == y: # y non ha vicini ancora non visitati
            S.pop()
```

```
18      return Vis
```

A prima vista anche questo algoritmo sembra avere una complessità di $O(n^2)$: infatti, nel while interno iteriamo nel caso peggiore $\deg(y)$ volte, dunque abbiamo per $2n-1$ iterazioni un costo di $O(\deg y) = O(n)$, e quindi $T(n) = O(n^2)$. Ma proviamo a studiarne il costo da un'altra prospettiva: prendiamo un singolo vertice $y$ del grafo e studiamo quanto costa questo vertice all'algoritmo.
Preso $y$, sappiamo che nel caso peggiore costerà al nostro algoritmo $O(\deg(y))$ (ossia quando tutti i suoi vicini sono già stati visitati). Dunque possiamo dire che ogni nodo $y$ ha costo $O(\deg(y))$. Ogni vicino di $y$ viene esaminato e rimosso da `y.adiacenti` una sola volta in tutta l'esecuzione, quindi il lavoro complessivo del while interno su $y$ è $O(\deg(y))$; sommando su tutti i nodi otteniamo $T(n) = O(\sum_{y \in V} \deg(y)) = O(m)$. Dato che però il nostro while ha costo minimo $O(n)$ ($n$ iterazioni di un while hanno infatti sempre un costo minimo di $O(n)$, che troviamo quando le operazioni all'interno del while hanno costo $O(1)$), possiamo dire che $\boldsymbol{T(n) = O(n+m)}$.
Questo ragionamento sul codice visto prima non funziona invece: infatti, per ogni vicino di $y$ li dobbiamo scorrere uno ad uno ogni volta, anche se li abbiamo già visti, portando il costo del singolo nodo a $O(\deg^2(y))$.

**N.B.** Negli algoritmi futuri useremo quasi sempre come DFS non questo ma l'altro algoritmo in costo $O(n^2)$ per semplicità.

### 1.6.1 DFS nei grafi non connessi

**Definizione 1.9 – Componente**

In un grafo non connesso le sue **componenti** sono i suoi sottografi connessi *massimali*, ossia non contenuti in nessun sottografo connesso più grande.




Ad esempio, in questo grafo vi sono due componenti: una formata dai nodi 1, 4 e 5, e una formata dai nodi 2, 3 e 6.

Vogliamo modificare ora la DFS per creare un nuovo algoritmo che ritorni l'array `Comp` lungo n, dove all'indice i vi è segnato il numero della componente in cui si trova il nodo $i$.

```
DFS_Componenti(G):
    n = numero di nodi di G
    Comp = [0] * n
    x = nodo qualsiasi
    contatore = 0
    while x != -1:
```

```
        contatore ++
        S = stack nuovo
        S.push(x)
        Comp[x] = contatore
        while S!= 0:
            y = S.top()
            if esiste z vicino di y tale che Comp[z] == 0:
                Comp[z] = contatore
                S.push(z)
            else:
                S.pop()
        x = -1
        for i in range(n):
            if Comp[i] == 0:
                x = i
                break
    return Comp
```

Riprendendo il grafo non connesso visto sopra, nella prima iterazione del ciclo while avremmo `Comp = [1,0,0,1,1,0]`, e alla seconda e ultima iterazione avremmo invece `Comp = [1,2,2,1,1,2]`.

### 1.6.2 DFS Ricorsivo

L'algoritmo DFS può essere riscritto anche come algoritmo ricorsivo:

```
DFS_Ricorsivo(G,x,Vis):
    Vis[x] = 1
    for y adiacente a x:
        if Vis[y] == 0:
            DFS_Ricorsivo(G,y,Vis)
```

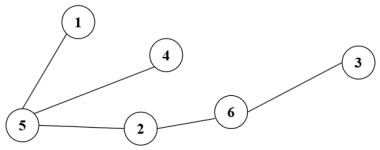
La complessità di questo algoritmo rimane $\mathbf{O(n+m)}$ per lo stesso ragionamento già visto per il `DFS_Alternativo`.

### 1.6.3 Alberi di ricerca

**Definizione 1.10 – Albero di ricerca**

Un **albero di ricerca** è un sottografo aciclico e connesso generato durante una visita (come la DFS) formato solo da archi che portano alla scoperta di nuovi nodi. Ha come **radice** un nodo del grafo e include quindi solo tutti i nodi raggiungibili da tale vertice.

Un albero di ricerca si memorizza tramite **vettore dei padri**, ossia un array di lunghezza n dove all'i-esima posizione si trova il nodo da cui parte un arco per arrivare al nodo $i$-esimo.




Ad esempio, in questo albero di ricerca il vettore padri sarebbe `P = [5,5,6,5,5,2]`.

Notiamo come la radice nel vettore padri abbia come valore sé stessa: dunque per trovarla basta solo trovare l'indice i del vettore tale che `P[i] == i`. Notiamo anche che non tutti i valori sono presenti: infatti, mancano l'1, il 4 e il 3. Questo succederà in ogni albero di ricerca: infatti, essendo aciclico, ci sarà sempre almeno un nodo che non avrà alcun "figlio", ossia a cui arriviamo da un suo arco. Questi nodi prendono il nome di **foglie**.

Studiamo ora un algoritmo per trovare la **distanza dalla radice** di un nodo $x$ in un albero di ricerca. La distanza dalla radice è il numero di archi del cammino che porta dal nodo $x$ alla radice.

```
Distanza_Radice(P,x):
    con = 0
    while P[x] != x:
        con ++
        x = P[x]
    return con
```

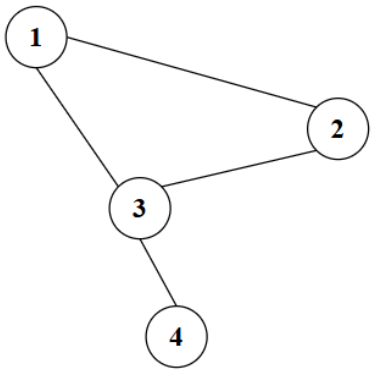
Il costo di questo algoritmo è $O(h)$, dove $h$ indica l'**altezza** dell'albero, ossia la massima distanza tra la radice e una delle foglie. Se ci troviamo in un albero degenere (ossia un albero lineare, dove l'altezza è pari a $n-1$), il costo può arrivare a $O(n)$.

### 1.6.4 Problema degli esami

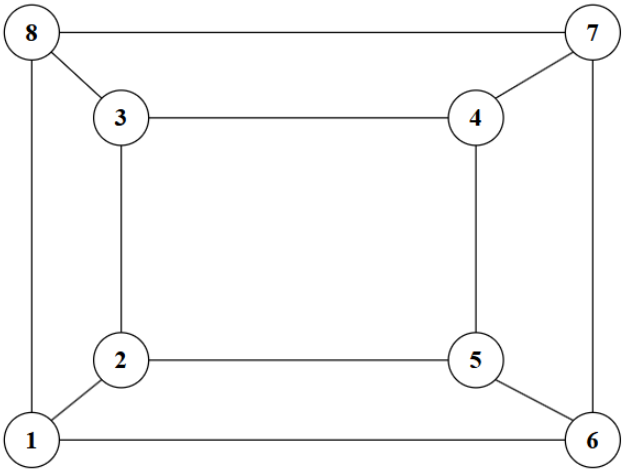
Vogliamo risolvere il seguente problema:

"Presi n esami e m studenti, vogliamo dividere gli esami in due settimane in modo tale che nessuno studente abbia 2 esami la stessa settimana. **N.B.** Uno studente ha sempre due esami da affrontare."

Trasformiamo questo problema in un problema con i grafi: i vertici saranno i diversi esami, mentre gli archi che li collegano saranno gli studenti. Se ad esempio abbiamo 4 studenti e 4 esami, un grafo potrebbe essere il seguente:




Essendoci in questo caso un ciclo a 3, notiamo subito come il problema non sia risolvibile: infatti, i nodi 1, 2 e 3 non possono mai stare insieme, ma avendo solo 2 settimane questo non è possibile. Vediamo ora un altro esempio invece:




Partiamo col dire che, preso un nodo $x$, se esiste una soluzione a questo problema sicuramente esiste una soluzione con $x$ nella prima settimana. Questo è ovvio dato che se esiste una soluzione con $x$ nella seconda, basta scambiare tutti i nodi tra una settimana e l'altra e avremo una soluzione con $x$ nella prima settimana. Dunque possiamo partire col posizionare il nodo 1 nella settimana 1 ad esempio. Ora, tramite una DFS, visitiamo tutti i nodi del grafo e li mettiamo alternati nelle diverse settimane. Troveremo quindi i nodi $\{1, 3, 5, 7\}$ nella prima settimana e i nodi $\{2, 4, 6, 8\}$ nella seconda settimana. Questo però non basta: dobbiamo verificare che per ogni arco nel grafo che collega due nodi questi non siano nella stessa settimana. La DFS controlla solo alcuni archi, non tutti, quindi questo controllo è fondamentale. Nel nostro caso la soluzione data non è

contraddetta da nessun arco, dunque è corretta. Vediamo come si traduce questo ragionamento in codice, supponendo per semplicità che il grafo sia connesso.

```
Esami(G): // G grafo connesso
    n = numero di nodi di G
    x = nodo qualsiasi
    Part = [0] * n
    Part[x] = 1
    S = stack nuovo
    S.push(x)
    while S != 0:
        y = S.top()
        if esiste un vicino z di y tale che Part[z] == 0:
            Part[z] = 3 - Part[y] # In questo modo alterniamo 1 e 2
            S.push(z)
        else:
            S.pop()
    for arco uv in G:
        if Part[u] == Part[v]:
            return false
    return Part
```

### 1.6.5 DFS per i grafi diretti

L'algoritmo della DFS per i grafi diretti rimane praticamente invariato, se non per la condizione dell'`if`, che diventa `if esiste un vicino con arco (y,z) e Vis[z] == 0`. In questi grafi, però, gli alberi di ricerca prendono il nome di **arborescenza**: è un grafo aciclico connesso che ha per ogni nodo un solo cammino possibile dalla radice a esso.

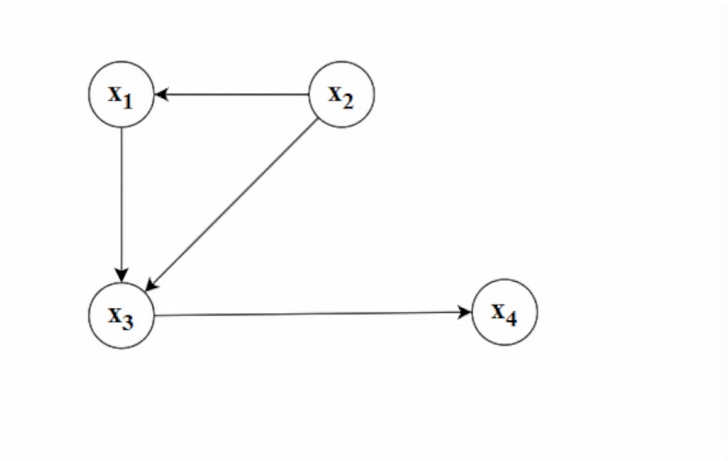
Tramite questi grafi connessi si possono risolvere problemi di questo genere:

"Un processo è diviso in una serie di lavoretti $x_1, \dots, x_n$ con delle dipendenze tra loro, del tipo $x_i$ va prima di $x_j$. Trovare un ordine dei lavoretti che rispetti tutte le dipendenze."

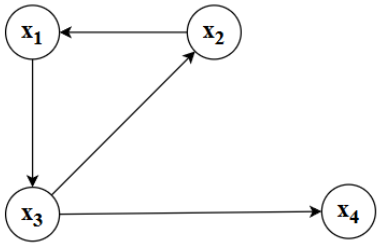
I vertici saranno i diversi lavoretti, mentre gli archi sono le dipendenze. Ad esempio, se $x_1$ viene prima di $x_2$, lo indicheremo con un arco diretto da $x_2$ a $x_1$ (indichiamo con la freccia chi viene prima). Se ad esempio abbiamo la lista di lavoretti $x_1, x_2, x_3, x_4$ con queste dipendenze

- $x_1$ prima di $x_2$
- $x_3$ prima di $x_2$
- $x_3$ prima di $x_1$
- $x_4$ prima di $x_3$

allora il grafo sarà di questo tipo:




In questo caso la soluzione sarebbe $x_4$ — $x_3$ — $x_1$ — $x_2$. Questo ordine viene detto **ordine topologico del grafo**. È importante capire che se il nostro grafo avesse avuto un **ciclo diretto** (ossia una passeggiata di almeno un arco che segue il verso degli archi e non ripete vertici, tranne il primo che coincide con l'ultimo), allora non avremmo avuto soluzione.




Ad esempio, in questo grafo non è possibile trovare un ordine topologico: i nodi $x_1$, $x_2$ e $x_3$ si trovano in un ciclo diretto, dunque non è possibile stabilire chi debba venire prima.

**Proposizione 1.5**

Preso un grafo $G$ diretto, se **non ci sono cicli diretti** $\implies$ **esiste un vertice** $v$ **senza archi uscenti**.

*Dimostrazione*

Per dimostrare questa proposizione dimostriamo che se ogni vertice ha archi uscenti allora c'è un ciclo diretto. Questo ha senso perché dimostrare che $A \implies B$ è uguale a dimostrare che $\neg A \impliedby \neg B$.

Prendiamo il seguente codice:

```
Trova_Ciclo_Diretto(G): # In G ogni nodo ha un arco uscente
    L = lista vuota
    x = nodo qualsiasi
```

```
    L.append(x)
    next = x.neighbors[0]
    while (next not in L):
        L.append(next)
        u = next.neighbors[0]
        next = u
    while L[0] != next:
        L.remove(L[0])
    return L
```

Poiché tutti i nodi hanno un arco uscente, `x.neighbors[0]` è sempre un'operazione sensata. La lista che ci verrà restituita dall'algoritmo è un cammino diretto il cui ultimo nodo ha un arco verso il primo: chiudendolo con quell'arco otteniamo un ciclo diretto. Dunque, preso un grafo diretto $G$ con tutti i vertici aventi almeno un arco uscente, questo ha sicuramente un ciclo diretto.
■

Ora che abbiamo ben compreso la DFS, vedremo una serie di problemi risolvibili tramite essa.

### 1.6.6 Trovare un ordine topologico in un grafo diretto aciclico

Vogliamo ora invece trovare un algoritmo che trovi un ordine topologico di un grafo diretto aciclico. Per la Proposizione 1.5 sappiamo che esiste sicuramente un vertice senza archi uscenti. Prendiamo questo vertice, lo segniamo, e lo eliminiamo dal grafo. A questo punto, dato che il grafo non ha cicli diretti, sicuramente vi è un altro nodo che non ha più archi uscenti. Lo prendiamo, lo segniamo e lo cancelliamo, e continuiamo così fino ad aver ricavato un ordine topologico. Se vi sono più nodi senza archi uscenti nello stesso momento basta prendere uno dei nodi in modo casuale: infatti, in uno stesso grafo possono esserci anche più ordini topologici diversi.
Vediamo il codice:

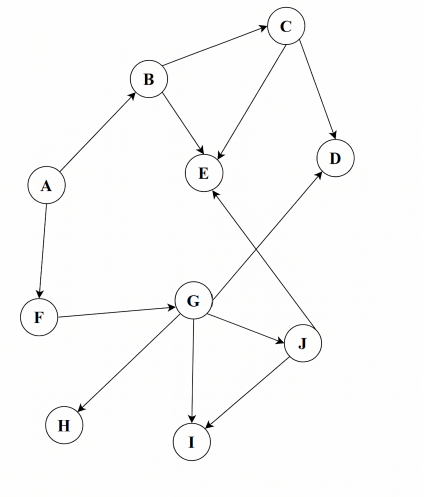
```
Ordine_Topologico(G): # G è un grafo diretto aciclico
    n = numero di nodi di G
    L = []
    while G != 0:
        v = vertice senza archi uscenti
        L.append(v)
        cancellare v da G
    return L
```

**N.B.** Questo algoritmo è un algoritmo distruttivo per il grafo: alla fine $G$ sarà vuoto, dunque il grafo va prima salvato per poter eseguire questo algoritmo.

Il costo computazionale di questo algoritmo è $\mathbf{O(n(n+m))}$. Infatti, il ciclo while esegue $O(n)$ iterazioni, e trovare un nodo senza archi uscenti costa $O(n)$, mentre eliminarlo costa $O(m)$ (dato che devo eliminare quel nodo da tutte le liste di adiacenza degli altri vertici). Dunque il costo diventa $O(n(n+m))$.

Il problema è che questo costo è veramente elevato (almeno $O(n^2)$), dunque cerchiamo di studiare un algoritmo che trovi un ordine topologico di un grafo diretto aciclico in tempo $O(n+m)$.

Partiamo sempre dalla DFS, ma aggiungiamo questa volta un contatore che incrementa ogni volta che visitiamo un vertice non ancora visto. Assegniamo poi ad ogni nodo $v$ due valori: $\boldsymbol{t(v)}$ è il valore del **contatore quando visitiamo il nodo** $v$, mentre $\boldsymbol{T(v)}$ è il valore del **contatore quando eliminiamo quel nodo dallo stack**.




Proviamo ad esempio a calcolare i valori di $t(v)$ e $T(v)$ di alcuni nodi in questo grafo. Ipotizziamo una DFS che segue questo ordine: $A, B, C, D, E, F, G, H, I, J$. Dunque $t(A) = 1, t(B) = 2, \ldots, t(J) = 10$, ossia ogni nodo $v$ ha come valore $t(v)$ la sua posizione nell'ordine dei nodi visitati. Per quanto riguarda i valori $T(v)$, troviamo ad esempio che $T(D) = 4$: infatti, questo nodo viene inserito e subito eliminato dallo stack dato che non punta a nodi adiacenti non visitati. Abbiamo anche che $T(E) = T(C) = T(B) = 5$: infatti, una volta visitato $E$ questo viene subito rimosso dato che non ha nodi adiacenti non visitati, e vengono poi rimossi anche $C$ e $B$. Infine, troviamo che $T(J) = T(G) = T(F) = T(A) = 10$: questi sono i nodi che rimangono nello stack una volta che sono stati visitati tutti i nodi, e che vengono quindi rimossi per ultimi.

**Osservazione 1.1**

In una **foglia** $f$ abbiamo che $\boldsymbol{t(f) = T(f)}$, mentre nella **radice** $r$ troviamo che $\boldsymbol{t(r) = 1}$ e $\boldsymbol{T(r) = n}$ (se tutti i nodi sono raggiungibili da $r$).

Ad ogni nodo $v$ associamo l'intervallo $\boldsymbol{[t(v), T(v)]}$.

**Proposizione 1.6**

In un grafo diretto, preso un arco $uv$ al di fuori dell'albero di ricerca vi sono tre possibilità:

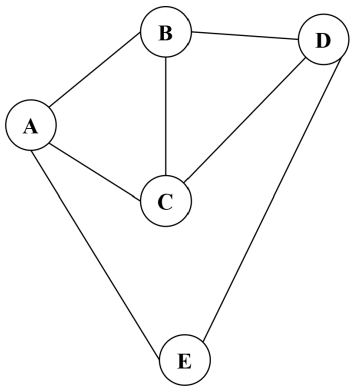
1) $[t(u), T(u)] \subseteq [t(v), T(v)]$ (**Arco all'indietro**, che va da un discendente a un antenato)
2) $[t(v), T(v)] \subseteq [t(u), T(u)]$ (**Arco in avanti**, che va da un antenato a un discendente)
3) $[t(u), T(u)] \cap [t(v), T(v)] = \varnothing$ (**Arco di attraversamento**, che collega due nodi dove nessuno è antenato dell'altro)

*Dimostrazione*

Vogliamo quindi dimostrare che è impossibile che gli intervalli di due nodi si sovrappongano, ossia che $t(u) < t(v) \leq T(u) < T(v)$.

Sappiamo che dire $t(u) < t(v)$ equivale a dire che $u$ è stato inserito nello stack prima di $v$, e dire che $T(u) < T(v)$ significa che $u$ è stato rimosso prima di $v$ dallo stack. Ma $t(u) < t(v) \leq T(u)$ significa che $v$ è stato inserito nello stack mentre $u$ era ancora presente: per la natura stessa di uno stack, che è una struttura dati LIFO (Last In First Out), $v$ deve uscire prima di $u$, ossia $T(v) \leq T(u)$, e questo contraddice $T(u) < T(v)$. Ciò significa che non esisteranno mai due nodi $u$ e $v$ tali che $t(u) < t(v) \leq T(u) < T(v)$. ■

Seguendo la stessa logica di questa dimostrazione, possiamo dire che preso un **arco** $(u, v)$ dell'albero di ricerca ci troviamo sicuramente nella **situazione 2)**. Infatti, sicuramente visitiamo prima il nodo $u$ del nodo $v$, dunque $t(u) < t(v)$, e dato che ci troviamo in uno stack sicuramente rimuoveremo prima $v$, dunque $T(v) \leq T(u)$.




È possibile usare questo stesso algoritmo anche nei grafi **non diretti**. In questo grafo ad esempio, seguendo come ordine del DFS $A, C, D, B, E$, il nodo $A$ ha come intervallo $[1, 5]$, mentre $D$ ha ad esempio $[3, 5]$.

Essendo gli archi non orientati, dei 3 casi visti nella Proposizione 1.6 ne abbiamo ora solo due:

1) $[t(u), T(u)] \subseteq [t(v), T(v)]$ (**Arco all'indietro**)
2) $[t(u), T(u)] \cap [t(v), T(v)] = \varnothing$ (**Arco di attraversamento**)

**Proposizione 1.7**

In un grafo non diretto connesso **non esistono archi di attraversamento al di fuori dell'albero di ricerca**.

*Dimostrazione*

Poniamo per assurdo che esista un arco $(u, v)$ tale che $[t(u), T(u)] \cap [t(v), T(v)] = \varnothing$. Assumiamo che $u$ venga nell'albero di ricerca prima di $v$, ossia che $t(u) < t(v)$. Dato che esiste un arco $(u, v)$, non è possibile che il nodo $u$ venga eliminato dallo stack prima della visita del nodo $v$. Infatti, per eliminare un nodo è necessario aver visitato tutti i suoi vicini, quindi prima di eliminare $u$ dobbiamo per forza di cose visitare $v$. Dunque sicuramente $t(v) \leq T(u)$, ossia $I_u \cap I_v \neq \varnothing$. ■

**Proposizione 1.8**

In un grafo connesso non diretto, $G$ ha un ciclo $\iff$ ogni DFS ha un arco all'indietro.

*Dimostrazione*

1° $G$ ha un ciclo $\implies$ ogni DFS ha un arco all'indietro
Sappiamo per la proposizione appena vista che ogni arco fuori dall'albero di ricerca in un grafo non diretto è all'indietro. Preso un ciclo, per definizione stessa dell'albero sicuramente non esiste un ciclo in questo (gli alberi sono infatti grafi aciclici). Dunque esiste sicuramente un arco del ciclo che non fa parte dell'albero, e dato che tutti gli archi fuori dall'albero sono all'indietro, questo è un arco all'indietro. ■

2° $G$ ha un ciclo $\impliedby$ ogni DFS ha un arco all'indietro
Sia $uv$ un arco all'indietro fuori dall'albero di ricerca con $u$ discendente di $v$. Sappiamo sicuramente che nell'albero esiste un cammino $P$ da $u$ a $v$, e dato che $uv$ non fa parte dell'albero questo chiude un ciclo. ■

**Proposizione 1.9**

In un grafo connesso diretto, $G$ ha un ciclo $\iff$ ogni DFS ha un arco all'indietro.

*Dimostrazione*

1° $G$ ha un ciclo $\implies$ ogni DFS ha un arco all'indietro
La dimostrazione purtroppo non è identica, perché qui non abbiamo più la certezza che tutti gli archi fuori dall'albero di ricerca siano all'indietro. Dunque dobbiamo fare una dimostrazione differente.
Supponiamo che se $C$ è un ciclo nel grafo, e $v$ è il primo vertice del ciclo visitato dal DFS, allora tutti i vertici del ciclo sono discendenti di $v$. Infatti sappiamo che tutti i vertici

oltre $v$ sono visitati prima che $v$ venga rimosso dallo stack, dunque possiamo dedurre che tutti i nodi sono discendenti di $v$. Se questo infatti per assurdo non fosse vero, vuol dire che avremmo un nodo $v_i$ sul ciclo non discendente di $v$, ossia che esiste un passo tra un vertice $v_j$ a un suo non discendente $v_{j+1}$ nel quale la discendenza quindi si perde. Sappiamo che $t(v_{j+1}) \leq T(v_j) \leq T(v)$. Questo significa che $v_{j+1}$ è stato inserito prima di aver rimosso $v$, dunque $v_{j+1}$ viene rimosso prima di $v$, dunque è un discendente di $v$. Dunque sicuramente non esistono nodi non discendenti di $v$ nel ciclo. Ma allora possiamo concludere che l'ultimo arco che collega l'ultimo nodo a $v$ e che chiude il ciclo è proprio un arco all'indietro, dato che collega un discendente con un antenato. ■

2° $G$ ha un ciclo $\Longleftarrow$ ogni DFS ha un arco all'indietro
La dimostrazione è identica a quella vista per la Proposizione 1.8. ■

Vediamo ora l'algoritmo per riempire gli array $T$ e $t$ in un grafo diretto connesso $G$:

```
Algoritmo(G,x):
    n = numero di nodi di G
    t = [-1] * n
    T = [-1] * n
    con = 1
    t[x] = 1
    Algoritmo_Ricorsivo(G,x,t,T,con)
```

dove `Algoritmo_Ricorsivo` è questa funzione ricorsiva:

```
Algoritmo_Ricorsivo(G,x,t,T,con):
    while esiste un nodo y con arco (x,y) e t[y] ==    Ora che abbiamo trovato che $r$
         è un'istanza legale di $R$ troviamo che questa soddisfa la dipendenza
        funzionale $X \longrightarrow Y$, dato che questa appartiene a $F^$ (
        ricordiamo che ogni dipendenza funzionale in $F^+$ è, per definizione,
        soddisfatta da qualsiasi istanza legale di $R$). Poiché $X \subseteq X^+$ (
        dato che $X \longrightarrow X$ è sempre soddisfatta), troviamo che $t_1[X] =
        t_2[X]$ e quindi, poiché $r$ soddisfa $X \longrightarrow Y$, anche $t_1[Y] =
        t_2[Y]$ è verificata. Ciò significa che $Y \subseteq X^+$ e, per il lemma di
        chiusura, troviamo proprio che $X \longrightarrow Y \in F^A$. $\blacksquare$
        -1:
        con ++
        t[y] = con
        con = Algoritmo_Ricorsivo(G,y,t,T,con)
    T[x] = con
    return con
```

**Proposizione 1.10**

In un ordine topologico $T$ di un grafo $G$ diretto e aciclico, calcolato con una DFS che raggiunge tutti i nodi, preso un arco $(u, v) \in E(G)$, $\boldsymbol{T(v) \leq T(u)}$.

*Dimostrazione*

Abbiamo solo due casi da analizzare: se $(u, v)$ è un arco in avanti o di attraversamento. Infatti, se abbiamo trovato un ordine topologico sicuramente in $G$ non è possibile trovare cicli, e per la Proposizione 1.9 sappiamo che se $G$ non ha cicli allora nessuna DFS sul grafo produce archi all'indietro, che altrimenti formerebbero un ciclo. Analizziamo quindi i seguenti casi:

- $(u, v)$ è un arco in avanti
  In questo caso allora abbiamo che $[t(v), T(v)] \subseteq [t(u), T(u)]$, dunque troviamo proprio che $T(v) \leq T(u)$.
- $(u, v)$ è un arco di attraversamento
  In questo caso allora possiamo sicuramente dire che la DFS abbia visitato prima $v$ e poi $u$, altrimenti, se avesse visto prima $u$, sarebbe passata per questo arco collegando subito anche $v$. Dunque, dato che $v$ è stato visitato prima e da lui non è possibile arrivare a $u$ (questo perché $(u, v)$ è un arco di attraversamento che quindi collega due "cugini", ma se da $v$ potessimo arrivare a $u$ allora questo sarebbe un antenato di $u$, cosa dunque impossibile), troviamo che $v$ esce dallo stack prima che entri $u$, ossia $T(v) < t(u) < T(u)$.

Concludiamo quindi che in ogni caso $T(v) \leq T(u)$. ■

Grazie a questa proposizione, è possibile ora trovare facilmente un ordine topologico in un grafo diretto aciclico. Infatti, basta prendere i nodi nell'ordine in cui escono dallo stack, cioè ordinati per $T$ crescente e, a parità di $T$, per ordine di uscita, per avere un ordine topologico dove gli archi vanno sempre da destra a sinistra. Infatti, questo ha ora senso dato che preso un arco $(x, y)$, è vero sempre che $T(y) \leq T(x)$. Vediamo come si traduce questo in codice:

```
OrdTop(G): // G grafo diretto aciclico
    n = numero di nodi di G
    L = [] // Lista dove inseriremo il nostro ordine topologico
    Vis = [0] * n
    for nodo v in G:
        if Vis[v] == 0:
            OrdTop_ric(G,v,Vis,L)
    return L
```

dove `OrdTop_ric` è questa funzione ricorsiva:

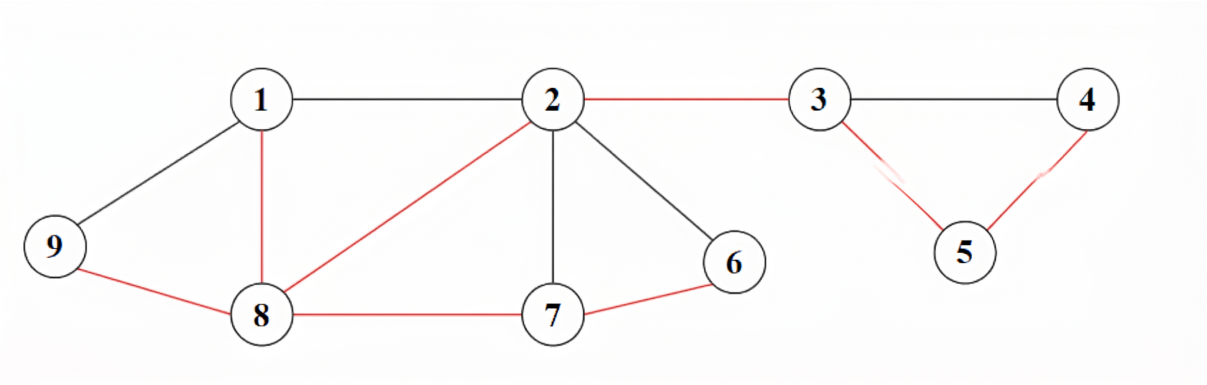
```
OrdTop_ric(G,v,Vis,L):
```

```
    Vis[v] = 1
    for nodo adiacente u a v:
        if Vis[u] == 0:
            OrdTop_ric(G,u,Vis,L)
    L.append(v)
```

Sembra strano però: non stiamo mai calcolando il valore $t$ o $T$ di nessun nodo. Questo però non è necessario: infatti, ogni volta che abbiamo un arco $(u, v)$ abbiamo detto che $v$ esce dallo stack prima di $u$, e infatti noi nel nostro algoritmo mettiamo sempre $v$ prima di $u$ nella lista (aggiungendolo dopo usando `append`), in modo da rispettare questa condizione sempre e senza dover calcolare il valore $T$ dei nodi.

### 1.6.7 DFS impossibili

Vediamo il seguente DFS:




Se proviamo a capire da quale nodo questo DFS parta, ci accorgiamo di una verità inevitabile: questo DFS **non può partire da NESSUN nodo**. Infatti, una volta che si arriva al nodo 8, da qualsiasi nodo si parta, ci si blocca: infatti dall'8 se andassimo al 7 allora dovremmo andare anche al 6 o al 2, cosa che nel nostro DFS non succede. Se invece andassimo al 9 dovremmo andare all'1 e viceversa, e se andassimo al 2 dovremmo andare al 7 e poi al 6 o viceversa. Notiamo quindi come i due archi problematici siano gli archi 67 e 91. Questi due archi collegano infatti in entrambi i casi due nodi di grado 1 (nell'albero di ricerca), e dato che questi sono per forza archi all'indietro (abbiamo visto come nei grafi non diretti tutti gli archi fuori dall'albero di ricerca siano archi all'indietro), troviamo che per forza un nodo è antenato dell'altro, ed essendo entrambi di grado 1, troviamo che uno dei due deve essere per forza la radice. Quindi troviamo che un nodo tra 6 e 7 è la radice, e anche uno tra 9 e 1 lo è: questo è ovviamente impossibile, e quindi capiamo subito che il DFS è impossibile.
Possiamo quindi ricapitolare che, se in un DFS ci sono due nodi $A$ e $B$ di grado 1 collegati da un arco fuori dal DFS e altri due nodi $C$ e $D$ con la stessa proprietà, il DFS è **impossibile**.

### 1.6.8 Pozzi universali

**Definizione 1.11 – Pozzo universale**

In un grafo diretto un **pozzo universale** è un nodo $x$ a cui tutti gli altri nodi **puntano** ma che **non punta** a nessun nodo.

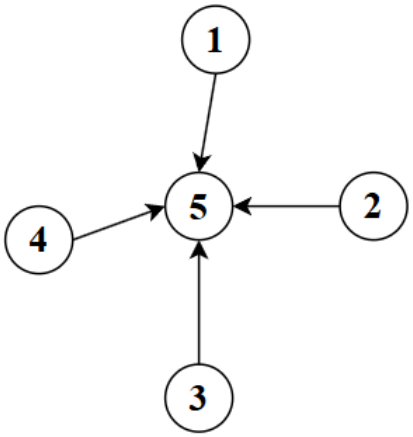



**Figura 1.1:** Esempio di pozzo universale

Vogliamo ora stabilire, preso un grafo diretto $G$ salvato tramite matrice di adiacenze, se **esiste un pozzo universale** (uno solo, dato che in ogni grafo può esistere al più un pozzo universale per la definizione stessa di pozzo) in $G$ in tempo $\mathbf{O(n)}$. Ricordiamo che in una matrice di adiacenze $(i, j) \in E(G)$ si traduce in $m[i, j] = 1$ (scriviamo quindi sulle colonne gli archi entranti di un nodo e sulle righe gli archi uscenti). Stiamo quindi cercando un nodo che abbia **una colonna di soli 1** (tranne che per la cella che si incrocia con sé stesso), e **una riga di soli 0**. Sembra facile: basta controllare ogni colonna di ogni nodo e ogni sua riga. Questo però ha costo totale $O(n^2)$ e non l'$O(n)$ richiesto. Per questo motivo cerchiamo un approccio diverso.
Notiamo che, presa una cella $(i, j)$, se $m[i, j] = 1$ allora possiamo subito eliminare $i$ dai possibili candidati al pozzo universale, dato che manda un arco in uscita, mentre se $m[i, j] = 0$ allora possiamo subito eliminare $j$ dai possibili candidati, dato che il nodo $i$ non punta a $j$. Dunque, con un controllo su una singola cella, in ogni caso riusciamo a eliminare un nodo dai possibili candidati ad essere pozzo universale. Ci bastano quindi solo $n - 1$ check per trovare un possibile pozzo universale, e ci servono poi solo $2n$ controlli su quel candidato per controllare che funzioni davvero da pozzo universale. Vediamo come si traduce questo in codice:

```
PozzoUniversale(m):
    n = numero di nodi del grafo
    candidato = 0
```

```
    for i in range(1,n):
        if m[candidato,i] == 1: // Il candidato ha un arco di uscita, non va bene
            candidato = i
    for i in range(n): // Check per controllare che candidato funzioni
        if m[i, candidato] == 0 and i != candidato: // Il candidato non ha un arco in
            entrata da i, non va bene
            return False
        if m[candidato,i] == 1: // Il candidato ha un arco in uscita, non va bene
            return False
    return candidato // Se supera i test è un vero pozzo universale
```

### 1.6.9 Ponti

**Definizione 1.12 – Ponte**

In un grafo connesso e non diretto, un arco è **ponte** se non appartiene a nessun ciclo.

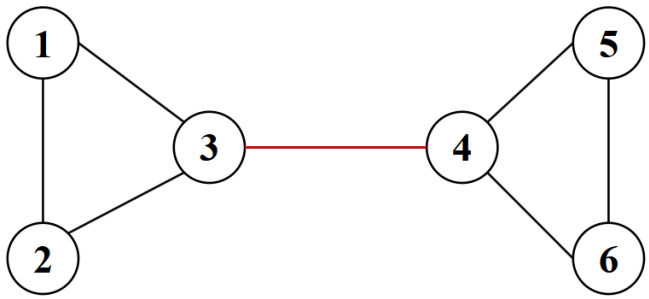



**Figura 1.2:** Esempio di ponte

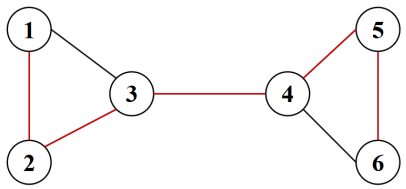
**Osservazione 1.2**

Preso un grafo $G$, $G$ è un albero $\implies$ tutti gli archi di $G$ sono ponti.

Questo è ovvio, dato che gli alberi sono aciclici.

Per verificare se un arco $xy$ sia un ponte, è necessario eliminarlo dal grafo ed eseguire una DFS da $x$ o da $y$ per verificare se esiste un cammino da $x$ a $y$. Se **questo cammino esiste** nonostante $xy$ sia stato eliminato, allora $xy$ **non è un ponte**: infatti questo sarebbe un secondo cammino da $x$ a $y$, che insieme all'arco $xy$ formerebbe un ciclo. Se invece **non troviamo un cammino** da $x$ a $y$, allora $xy$ **è un ponte**. Il costo di questa operazione è quindi il costo di una DFS, ossia $O(n + m)$, più il costo dell'eliminazione di un arco, ossia $O(n)$. Dunque il costo totale è $\mathbf{O(n + m)}$.

Se ora volessimo invece trovare tutti i ponti in un grafo $G$, potremmo pensare di eseguire questo algoritmo con tutti gli archi in $G$. Ciò però avrebbe un costo di $O(m(n+m))$, abbastanza elevato. Proviamo quindi a ragionare in modo diverso.
Sappiamo che se eseguiamo un DFS su $G$, troveremo un albero di ricerca $T$. Trovandoci in un grafo connesso non diretto, per la Proposizione 1.7 sappiamo che tutti gli archi fuori da $T$ sono all'indietro, e per la Proposizione 1.8 sappiamo che questi archi formano sempre dei cicli, dunque possiamo già concludere che **tutti gli archi fuori da $T$ non sono ponti**. Non ci resta ora che analizzare gli archi di $T$, che sappiamo essere $n-1$, e per farlo possiamo usare l'algoritmo visto prima. Arriviamo quindi a un costo totale di $\mathbf{O(n(n+m))}$, migliore rispetto al costo visto prima, dato che in un grafo connesso abbiamo $m \geq n-1$, ossia $m \in \Omega(n)$.
Vediamo un esempio per capire meglio questo procedimento.




Sappiamo già quindi che 13 e 46 non sono ponti, essendo fuori dall'albero di ricerca. 12 nemmeno lo è, perché eliminandolo riusciamo comunque a trovare un cammino tra 1 e 2 ($1-3-2$). Stesso discorso per gli archi 23, 45 e 56. L'unico che veramente è un ponte è proprio l'arco 34, che avevamo già visto nell'esempio in Figura 1.2.

Notiamo da questo esempio, però, una cosa molto interessante, ossia che da un nodo $x$ collegato a un nodo $y$ tramite un arco che non è un ponte esiste un altro arco all'indietro che parte da $y$ o da un suo discendente e arriva a un antenato di $x$. Ipotizziamo ad esempio che il nostro DFS parta dal nodo 1: se prendiamo l'arco 56, notiamo che l'arco all'indietro collega 6 a un antenato di 5: il nodo 4. Questo ci assicura già che anche 56 non può essere un ponte: infatti, se 6 è collegato a un antenato di 5, vuol dire che esiste un cammino che da 6 porta a 5 senza passare per l'arco 56, il che rende 56 non un ponte. Racchiudiamo questa idea ora in una proposizione e dimostriamola.

**Proposizione 1.11**

Preso un grafo $G$ connesso e non diretto, e $T$ un albero di ricerca derivante da un DFS su $G$, e $xy$ un arco di $T$, dove $y$ è un discendente di $x$, allora esiste un arco $uv \neq xy$, con $u \in T_y$ (dove $T_y$, o $tree(y)$, è il sottoalbero formato dai discendenti di $y$, dunque anche da $y$ stesso) e $v$ antenato di $x \iff xy$ non è un ponte.

*Dimostrazione*

1° Esiste un arco $uv \neq xy$, con $u \in T_y$ e $v$ antenato di $x \implies xy$ non è un ponte.
Per dimostrare che $xy$ non è un ponte, dobbiamo dimostrare che tolto questo esista comunque un cammino che porti da $x$ a $y$ nel grafo. Poiché $u \in T_y$, sappiamo che $u$ è discendente di $y$ e che quindi esiste un cammino da $y$ a $u$. Per lo stesso motivo, poiché $v$ è antenato di $x$, sicuramente esiste un cammino da $v$ a $x$. Dunque, passando per il cammino

da $y$ a $u$, per l'arco da $u$ a $v$, e per il cammino da $v$ a $x$, troviamo un cammino che collega $x$ a $y$, senza passare per l'arco $xy$. Possiamo quindi concludere che $xy$ non è un ponte. ■

2° Esiste un arco $uv \neq xy$, con $u \in T_y$ e $v$ antenato di $x \implies xy$ non è un ponte.
Non essendo $xy$ un ponte, sappiamo che esiste un ciclo $C$ che lo contiene. Vogliamo ora dimostrare che anche $uv$ è in $C$.
Sappiamo già che esiste un cammino $P$ che porta da $y$ a $x$ senza passare per $xy$ (banalmente, $P = C - xy$). Dato che $P$ parte da $y \in T_y$ e arriva a $x \notin T_y$, è necessario che lungo $P$ vi sia un arco $uv$ dove $u \in T_y$ e $v \notin T_y$. Ciò significa che $v$ è antenato di $u$, e dato che $v \notin T_y$ questo è anche antenato di $y$. Essendo quindi $v$ antenato di $y$ e diverso da $y$, troviamo che $v$ è antenato anche di $x$ (infatti $x$ è il padre di $y$, quindi ogni antenato di $y$ diverso da $y$ è $x$ stesso oppure un antenato di $x$). Abbiamo quindi trovato un arco $uv \neq xy$ (diverso da $xy$ dato che $uv \in P = C - xy$), con $u \in T_y$, e $v$ antenato di $x$. ■

Grazie a questa proposizione possiamo scrivere un algoritmo che trova tutti i ponti in un grafo in tempo $\mathbf{O(n + m)}$:

```
CercaPonti(G): // G è un grafo connesso e non diretto
    n = numero di nodi di G
    t = [0] * n
    z = nodo in G
    t[z] = 1
    c = 1
    Ponti = lista vuota
    for y adiacente a z:
      if t[y] == 0:
        b = CercaPonti_ric(G, t, y, z, c, Ponti)
        if b >= t[y]: // Controlla se l'arco zy è un ponte
          Ponti.add(zy)
    return Ponti
```

dove `CercaPonti_ric` è questa funzione ricorsiva:

```
CercaPonti_ric(G, t, y, z, c, Ponti):
    c++
    t[y] = c
    back = t[y]
    for v adiacente a y:
      if t[v] == 0: // v ancora da visitare
        b = CercaPonti_ric(G, t, v, y, c, Ponti) // Cerchiamo ponti da v
        if b >= t[v]: // Da v non si va più sopra di v
            Ponti.add(yv)
        if b < back: // Se andiamo più indietro di quanto salvato fino ad ora
            back = b
```

```
12      else: // v già visitato
13        if v != z: // Evitiamo di risalire l'arco dell'albero verso il padre z
14            if t[v] < back: // Se andiamo più indietro
15                back = t[v]
16    return back
```

Il ragionamento dietro questo codice si basa sul DFS che già conosciamo bene. Per ogni nodo $y$ visitato, verifichiamo quanto in alto riusciamo a risalire lungo l'albero di visita (ossia qual è il minimo $t$ raggiungibile). Effettuiamo questo controllo su tutti i suoi vicini $v$, distinguendo due casi:

- *Se il vicino $v$ non è stato ancora visitato:* esploriamo ricorsivamente il suo sottoalbero. Una volta calcolato il minimo tempo di scoperta $b$ raggiungibile da $v$ e dai suoi discendenti, verifichiamo se $b < t[v]$:
  - Se da $v$ si riesce a raggiungere $y$ o un antenato di $y$ (cioè $b < t[v]$), significa che l'arco $yv$ appartiene a un ciclo, quindi non è un ponte.
  - Se invece non è possibile risalire oltre $v$ ($b \geq t[v]$), l'arco $(y, v)$ è sicuro un ponte per la Proposizione 1.11.

  Aggiorniamo poi la variabile `back` del nodo $y$ se con il sottoalbero di $v$ siamo riusciti a raggiungere un nodo scoperto ancora prima ($b <$ `back`).
- *Se il vicino $v$ è già stato visitato:* verifichiamo di non star risalendo lungo l'arco dell'albero verso il padre $z$ ($v \neq z$). In tal caso, l'arco $yv$ è un arco all'indietro, dunque verifichiamo se $t[v] <$ `back` e, in caso affermativo, aggiorniamo il valore di `back`.

La funzione ricorsiva restituisce infine, per ciascun nodo, il minimo tempo di scoperta raggiungibile tramite archi dell'albero o al più un arco all'indietro dal suo sottoalbero.

### 1.6.10 Grafi fortemente connessi

**Definizione 1.13 – Grafo fortemente connesso**

Un grafo $G$ diretto è detto **fortemente connesso** se $\forall x, y \in V(G)$ esiste un cammino da $x$ a $y$ e da $y$ a $x$.

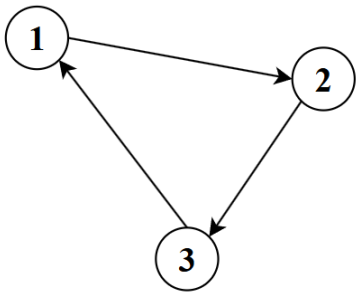



**Figura 1.3:** Esempio di grafo fortemente connesso

**Osservazione 1.3**

Preso un grafo diretto $G$ e un nodo $x$, in $G$ esiste sempre un **sottografo fortemente connesso contenente $x$** (banalmente, il sottografo che contiene solo $x$ è fortemente connesso).

**Definizione 1.14 – Componente fortemente connessa**

Preso un grafo $G$, una componente di questo grafo $H$ è detta **fortemente connessa** se è un sottografo fortemente connesso *massimale*, ossia non esiste un sottografo fortemente connesso $H'$ diverso da $H$ che contiene $H$ (è il più grande sottografo fortemente connesso che contiene quei nodi al suo interno).

Nell'esempio in Figura 1.3 tutto il ciclo è una componente fortemente connessa ad esempio.

**Proposizione 1.12**

Due componenti fortemente connesse **non possono avere un nodo in comune**.

*Dimostrazione*

Banalmente, se avessero un nodo in comune, questo permetterebbe di formare cammini da qualsiasi nodo della prima componente a qualsiasi nodo della seconda e viceversa. Ciò significa che esisterebbe una componente fortemente connessa che le contiene entrambe, ma per la stessa definizione di componente fortemente connessa questo non è possibile, dunque non è possibile che due componenti fortemente connesse abbiano un nodo in comune. ■

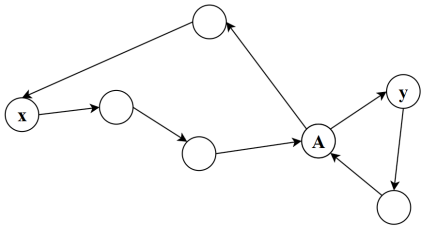
**Osservazione 1.4 – Osservazioni**

Preso un grafo $G$ diretto e due nodi $x, y \in G$:

1) Se $x$ e $y$ sono due nodi della stessa componente fortemente connessa, ciò **non implica che ci sia un ciclo che li contenga**
2) Se $x$ e $y$ sono due nodi facenti parte di un ciclo, allora $x$ e $y$ **fanno parte della stessa componente fortemente connessa**

*Dimostrazione*

1) Banalmente, non è detto che i due cammini siano disgiunti. Prendiamo ad esempio questo grafo:




Nonostante ci sia qui un cammino sia da $x$ a $y$ che viceversa, non c'è nessun ciclo che contiene sia $x$ che $y$ per colpa del nodo $A$. C'è infatti un ciclo che contiene $A$ e $x$ e uno che contiene $A$ e $y$, ma trovandosi questo in entrambi i cammini rompe il ciclo tra $x$ e $y$. ■

2) Ipotizziamo per assurdo che non facciano parte della stessa componente fortemente connessa, dunque $x$ si trova in una componente fortemente connessa $H$ diversa da quella che contiene $y$. Ma essendo il ciclo un sottografo fortemente connesso, il nodo $x$ crea un cammino da tutti i nodi in $H$ a $y$ e viceversa, dunque esiste un sottografo fortemente connesso $H'$ che contiene propriamente $H$ e il ciclo. Ma per la definizione di componente fortemente connessa, $H$ è massimale e non può essere contenuta propriamente da un altro sottografo fortemente connesso; dunque abbiamo trovato un assurdo, e quindi $x$ e $y$ sono nella stessa componente fortemente connessa. ■

**Proposizione 1.13 – Corollario**

Preso un grafo $G$ diretto, dato che ogni nodo in $G$ è parte di una componente fortemente connessa (per l'Osservazione 1.3), e dato che due componenti fortemente connesse sono disgiunte (per la Proposizione 1.12), possiamo concludere che le componenti fortemente connesse formano una **partizione dei nodi di $G$**.

Cerchiamo ora un algoritmo per trovare tutte le componenti fortemente connesse di un grafo diretto.

**Proposizione 1.14**

Preso un grafo $G$ diretto e un nodo $y \in G$, $G$ è **fortemente connesso** $\iff \forall x \in G$ **esiste un cammino da $x$ a $y$ e viceversa**.

*Dimostrazione*

1° $G$ è fortemente connesso $\implies \forall x \in G$ esiste un cammino da $x$ a $y$ e viceversa
Questa implicazione è ovvia e deriva proprio dalla definizione di grafo fortemente connesso. ■

2° $G$ è fortemente connesso $\impliedby \forall x \in G$ esiste un cammino da $x$ a $y$ e viceversa
Preso un nodo $u \in G$, sappiamo sicuramente che esiste un cammino $P$ da $x$ a $y$ e un cammino $Q$ da $y$ a $u$, dunque esiste anche un cammino da $x$ a $u$ ($P + Q$ è una passeggiata, non un cammino, ma per la Proposizione 1.3 sappiamo che esiste anche un cammino che collega questi due nodi). Allo stesso modo è possibile trovare un cammino da $u$ a $x$. Dato che questo vale per ogni coppia di nodi $(u, x)$ in $G$, possiamo concludere che $G$ è fortemente connesso. ■

**Definizione 1.15 – Contrarre un sottoinsieme**

Preso un grafo diretto $G$ e un sottoinsieme $X$ di nodi, **contrarre $X$** significa:

1° **Sostituire** il sottoinsieme $X$ con un nuovo nodo $v_X$

2° $\forall$ arco $(x, y)$ con $x \in X$ e $y \notin X$ aggiungere l'arco $(v_X, y)$ e viceversa

Il nuovo grafo formato dalla contrazione di $G$ viene denominato come $G/X$.

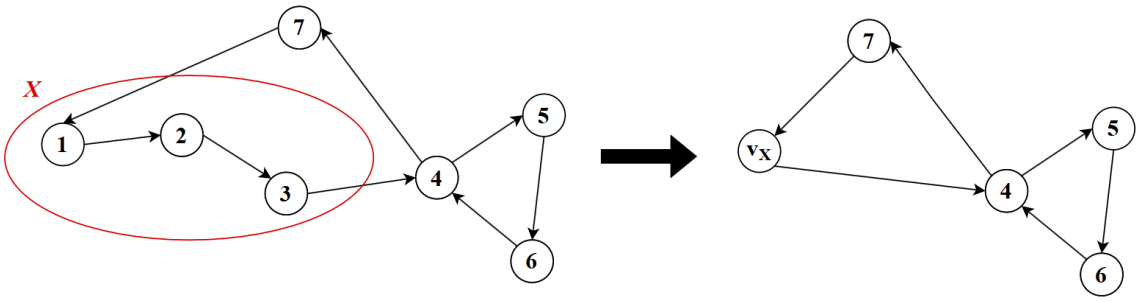



**Figura 1.4:** Esempio di sottoinsieme $X$ contratto

**Proposizione 1.15**

Preso un grafo $G$ fortemente connesso e $H$ un sottografo di $G$, $G/V(H)$ **è fortemente connesso**.

*Dimostrazione*

Sia $x$ un vertice di $H$ e $P$ un cammino da $y \in V(G/V(H))$ a $x$ in $G$ (sicuramente c'è questo cammino essendo $G$ fortemente connesso). Sia $\bar{P}$ il pezzo di $P$ che va da $y$ fino al 1° vertice che incontra di $H$. Dunque $\bar{P}$ in $G/V(H)$ è un cammino da $y$ a $v_H$ (scriviamo $v_H$ e non $v_{V(H)}$ per semplicità). Dunque in $G/V(H)$ esiste $\forall y \in V(G/V(H))$ un cammino da $y$ a $v_H$. Allo stesso modo, preso un cammino $Q$ in $G$ da $x$ a $y$ e il pezzo $\bar{Q}$ di $Q$ che parte dall'ultimo vertice di $H$ che incontra, $\bar{Q}$ in $G/V(H)$ è un cammino da $v_H$ a $y$. Quindi ogni $y$ raggiunge $v_H$ e viceversa, e per la Proposizione 1.14 $G/V(H)$ è fortemente connesso. ■

**Osservazione 1.5**

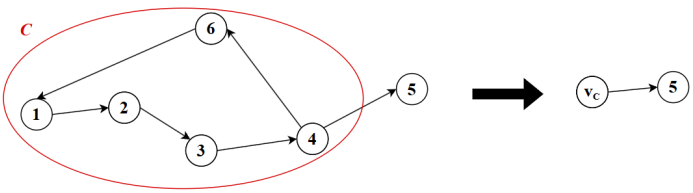
Preso un grafo $G$ diretto, esiste sempre un sottoinsieme $X \subseteq G$ che renda $G/X$ fortemente connesso.

Questo è sempre vero perché banalmente, prendendo $X = V(G)$, $G/X$ altro non è che un grafo con un solo nodo $v_X$, e un grafo con un solo nodo è per definizione fortemente connesso.

Grazie a tutte queste proposizioni e osservazioni riusciamo ad arrivare a un algoritmo che trovi tutte le componenti fortemente connesse di un grafo:

```
Componenti_FC(G): // G grafo diretto
    if esiste ciclo diretto C in G:
        H = G/C // Contraiamo C
        Componenti = Componenti_FC(H) // Troviamo le componenti del grafo contratto in
             modo ricorsivo
        for V in Componenti: // Cerchiamo vc e lo sostituiamo con i nodi di C
            if vc in V:
                Componenti.remove(V)
                Componenti.add((V - {vc}) U V(C))
                break
        return Componenti
    else: // Se non esiste un ciclo diretto in G allora le componenti sono i singoli
        nodi
        return {{v} per ogni v in V(G)}
```

Vediamo come funziona questo algoritmo con un esempio. Preso il grafo



troviamo prima il ciclo C formato dai nodi $1, 2, 3, 4$ e $6$, lo trasformiamo in $v_C$, e cerchiamo altri cicli in modo ricorsivo. Non trovandoli, la chiamata ricorsiva ritorna $\{\{v_C\}, \{5\}\}$, e il `for` trasforma questo insieme in $\{\{1, 2, 3, 4, 6\}, \{5\}\}$. Dunque troviamo che il grafo ha solo 2 componenti fortemente connesse: $\{1, 2, 3, 4, 6\}$ e $\{5\}$.

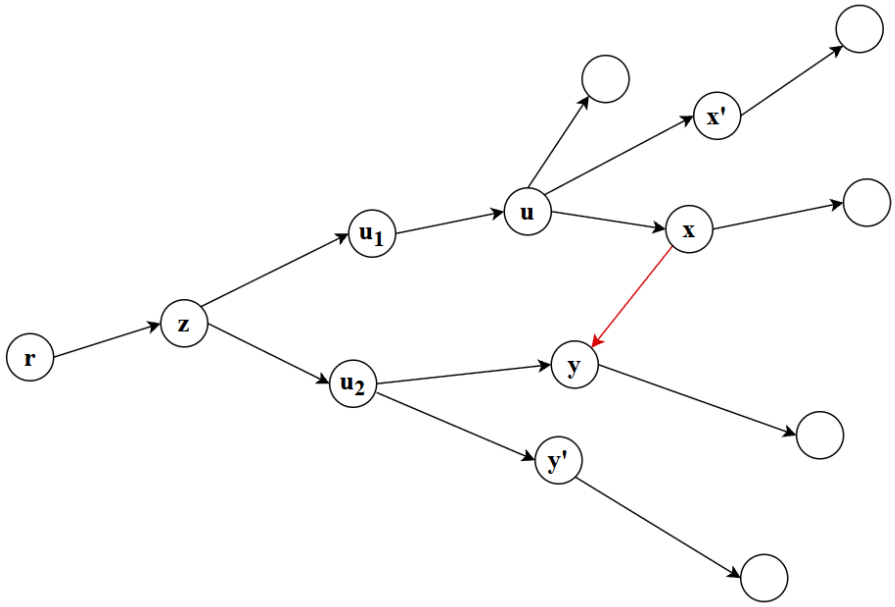
Per quanto riguarda il costo di questo algoritmo, sappiamo che la ricerca di un ciclo in un grafo richiede una DFS e quindi un costo di $O(n + m)$, e anche la contrazione e l'espansione di un ciclo nel grafo possono arrivare a costare $O(n + m)$. Andando in modo ricorsivo, è possibile dover effettuare queste operazioni in totale $O(n)$ volte, arrivando a un costo di $\mathbf{O(n(n + m))}$. Cerchiamo però ora di arrivare a un algoritmo con un costo computazionale $O(n + m)$.

**Proposizione 1.16**

Preso un grafo diretto $G$, un'arborescenza di questo grafo $T$ e un nodo $u \in V(G)$, $\exists$ un arco di attraversamento $(x, y) \in E(G)$ con $x \in T_u$ e $y \notin T_u \implies \exists$ l'antenato comune più vicino $z$ di $x$ e $y$ che è anche antenato di $u$, ed esistono due figli $u_1$ e $u_2$ di $z$ tali che $x \in T_{u_1}$ e $y \in T_{u_2}$ e, $\forall x' \in T_{u_1}$ e $\forall y' \in T_{u_2}$, $t(x') > t(y')$ (ossia tutti i nodi nel sottoalbero $T_{u_2}$ sono visitati prima di quelli del sottoalbero $T_{u_1}$).

*Dimostrazione*

Cerchiamo prima di tutto di capire visivamente come si presenta questa proposizione (vediamo l'arborescenza $T$ rappresentata, non tutto il grafo $G$, fatta eccezione per l'arco $(x, y) \in E(G)$ e $\notin E(T)$):



Per quanto riguarda la proposizione vera e propria, sappiamo che $z$ sicuramente esiste, dato che $x$ e $y$ hanno in comune come antenato almeno la radice dell'arborescenza. Questo $z$ è sicuramente anche antenato di $u$, perché se fosse suo discendente avremmo che $y \in T_z \subseteq T_u$, ossia $y \in T_u$ che per ipotesi non è vero. Essendo poi $z$ l'antenato comune più vicino, da $z$ i due nodi $x$ e $y$ seguono sicuramente due percorsi differenti, e chiamiamo quindi $u_1$ il primo nodo sul percorso che porta a $x$ e $u_2$ il primo nodo sul percorso che porta a $y$. Ora, essendo $T$ un'arborescenza, questa deriva da un DFS. Dato che esiste l'arco $(x, y)$, sicuramente $x$ è stato visitato quando $y$ era già stato visitato, altrimenti $y$ sarebbe stato scoperto durante la visita di $x$, diventando un suo discendente, e $(x, y)$ sarebbe un arco dell'albero o in avanti, non di attraversamento. Ma se $x$ è stato visitato dopo di $y$, significa che tutto il sottoalbero $T_{u_1}$ è stato visitato dopo del sottoalbero $T_{u_2}$, proprio per come funziona l'algoritmo di un DFS. Dunque $\forall x' \in T_{u_1}$ e $\forall y' \in T_{u_2}$ si ha che $t(y') < t(x')$. ■

**Definizione 1.16 – C-radice**

Presa una componente fortemente connessa $C$ di un grafo $G$ e un'arborescenza $T$ di un DFS su tale grafo, la **c-radice** di $C$ è il primo nodo di $C$ visitato nel DFS.

Vediamo questo grafo come esempio per capire cosa sono le c-radici. In rosso vi è un'arborescenza $T$ di questo grafo, con i valori di visita $t$ di ogni nodo segnati al loro interno. Presa dunque ogni componente fortemente connessa del grafo, la c-radice di ogni componente sarà il nodo con il minor valore di visita all'interno di tale componente. In $C_1$ quindi sarà il nodo 1, in $C_2$ sarà il nodo 3 e in $C_3$ il nodo 6.

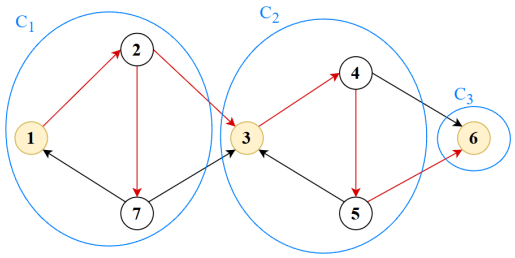



**Figura 1.5**

**Proposizione 1.17**

Preso $u$ una c-radice di una componente fortemente connessa:

1) $\boldsymbol{C(u) \subseteq T_u}$ (dove $C(u)$ indica la componente fortemente connessa a cui appartiene $u$)
2) Prese le c-radici $u_1, \ldots, u_k \in T_u$, $\boldsymbol{V(T_u) = C(u_1) \cup \ldots \cup C(u_k)}$

*Dimostrazione*

1) Supponiamo per assurdo che $C(u) \nsubseteq T_u$. Questo significa che $\exists y \notin T_u \wedge y \in C(u)$. Preso

un cammino da $u$ a $y$, sia $(x, y')$ il primo arco del cammino che esce da $T_u$ ($x \in T_u$, $y' \notin T_u$): anche $y' \in C(u)$, perché da $u$ si arriva a $y'$ e da $y'$ si arriva a $y$ e quindi a $u$. Sostituendo $y$ con $y'$, possiamo quindi supporre che esista un arco $(x, y)$ con $x \in T_u$, $y \notin T_u$ e $y \in C(u)$. Sicuramente sappiamo che $(x, y)$ non è un arco dell'albero né avanti, perché altrimenti avremmo $y \in T_x$ e quindi $y \in T_u$, dunque può essere solo di attraversamento o all'indietro. Analizziamo entrambi i casi:

- $(x, y)$ è di attraversamento
  In questo caso ci troviamo nelle esatte condizioni della Proposizione 1.16. Dunque sappiamo che $y$ viene visitato prima di $u$ nell'arborescenza, e dato che $y \in C(u)$ troviamo che $u$ non può essere una c-radice essendoci $y$ prima di lui. Ma per ipotesi sappiamo che $u$ è una c-radice, dunque concludiamo che $(x, y)$ non può essere un arco di attraversamento.
- $(x, y)$ è all'indietro
  In questo caso abbiamo quindi che $y$ è un antenato di $x$ ma non un discendente di $u$, dunque $y$ è anche un antenato di $u$. Ma per essere antenato di $u$ deve esser stato visitato prima di $u$, e dato che $y \in C(u)$, troviamo che $u$ non può essere una c-radice, trovando nuovamente una contraddizione con l'ipotesi iniziale.

Dato che $(x, y)$ non è un arco né di attraversamento, né all'indietro, né in avanti concludiamo che $(x, y)$ non esiste, dunque $C(u) \subseteq T_u$. ■

2) $\forall u_i$ sappiamo che $C(u_i) \subseteq T_u$ per 1) (infatti $C(u_i) \subseteq T_{u_i} \subseteq T_u$). Supponiamo ora per assurdo che 2) sia falso. Dunque sappiamo che $\exists w \in T_u$ con la c-radice $z$ di $C(w)$ che non appartiene a $T_u$ ($z \notin T_u$). $z$ è quindi sicuramente un antenato di $w$, ma è anche antenato di $u$ dato che $z \notin T_u$ e $z$ e $u$ si trovano su uno stesso cammino, essendo entrambi antenati di $w$. Dato che $z \in C(w)$, sappiamo che esiste un cammino da $z$ a $w$, e dato che $z$ è antenato di $u$ esiste un cammino tra $z$ e $u$, e dato che $u$ è antenato di $w$ esiste un cammino da $u$ a $w$. Possiamo quindi sempre trovare un cammino che parta da un nodo $z, w$ o $u$ e che arrivi a uno di questi tre nodi, dunque questi tre nodi si trovano nella stessa componente fortemente connessa. Ma dato che $z$ è un antenato di $u$, in questa componente fortemente connessa sicuramente $u$ non è una c-radice, e dato che per ipotesi $u$ è una c-radice, abbiamo trovato un assurdo. Concludiamo quindi che $\nexists w \in T_u$ con c-radice $z$ di $C(w)$ che non appartiene a $T_u$, ossia che $V(T_u) = C(u_1) \cup \ldots \cup C(u_k)$. ■

Vediamo ora un primo algoritmo ricorsivo per trovare tutte le componenti fortemente connesse di un grafo basato sulle c-radici:

```
1 DFS_SCC_beta(G,u,Open,Output,Vis): // G grafo diretto, Open stack nuovo e Output
      insieme con tutte le componenti fortemente connesse
2     Vis[u] = 1
3     Open.push(u)
4     for v adiacente a u:
5         if Vis[v] == 0:
```

```
            DFS_SCC_beta(G,v,Open,Output,Vis)
    if u è una c-radice: // Ipotizziamo di avere un modo per verificare che un nodo
        sia una c-radice
        C = []
        do:
            w = Open.pop()
            C.append(w)
        while w != u // Tutti i nodi nello stack prima della c-radice fanno parte
            della stessa componente fortemente connessa
        Output.add(C)
```

Cerchiamo di capire come funziona questo algoritmo dandogli in input il grafo visto in Figura 1.5. In particolare, studiamo come ad ogni chiamata ricorsiva si modificano l'insieme Output e lo stack Open.

1° Chiamata su 1: `Output = {}` e `Open = [1]`

2° Chiamata su 2: `Output = {}` e `Open = [2,1]` (la testa dello stack è l'elemento più a sinistra, in questo caso 2)

3° Chiamata su 3: `Output = {}` e `Open = [3,2,1]`

4° Chiamata su 4: `Output = {}` e `Open = [4,3,2,1]`

5° Chiamata su 5: `Output = {}` e `Open = [5,4,3,2,1]`

6° Chiamata su 6: `Output = {}` e `Open = [6,5,4,3,2,1]`

7° Fine chiamata su 6: `Output = {{6}}` e `Open = [5,4,3,2,1]` (6 è una c-radice)

8° Fine chiamata su 5: `Output = {{6}}` e `Open = [5,4,3,2,1]`

9° Fine chiamata su 4: `Output = {{6}}` e `Open = [5,4,3,2,1]`

10° Fine chiamata su 3: `Output = {{6}, {5,4,3}}` e `Open = [2,1]` (3 è una c-radice e si porta con sé 5 e 4 che fanno parte della sua stessa componente fortemente connessa)

11° Chiamata su 7: `Output = {{6}, {5,4,3}}` e `Open = [7,2,1]`

12° Fine chiamata su 7: `Output = {{6}, {5,4,3}}` e `Open = [7,2,1]`

13° Fine chiamata su 2: `Output = {{6}, {5,4,3}}` e `Open = [7,2,1]`

14° Fine chiamata su 1: `Output = {{6}, {5,4,3}, {7,2,1}}` e `Open = []` (1 è una c-radice e si porta con sé 7 e 2 che fanno parte della sua stessa componente fortemente connessa)

15° Ritorniamo `Output = {{6}, {5,4,3}, {7,2,1}}`

Dato che l'algoritmo funziona, non ci resta che cercare un modo per verificare se un nodo è una c-radice o no.

**Proposizione 1.18**

Un nodo $u$ **non è una c-radice** $\iff$ nella chiamata ricorsiva di `DFS_SCC_beta` sul nodo $u$ viene attraversato un arco $(v, w)$ tale che $w$ è già stato visitato ($\boldsymbol{t(w) < t(u)}$) ma la componente di $w$ ancora non è stata scelta ($\boldsymbol{w \in}$ `Open`).

*Dimostrazione*

1° $u$ non è una c-radice $\implies$ esiste un arco $(v, w)$ tale che $t(w) < t(u)$ e $w \in$ `Open`
Sappiamo che $\exists \alpha$ c-radice di $C(u)$. Dunque $\alpha$ è un antenato di $u$ ma esiste anche un cammino da $u$ a $\alpha$, trovandosi questi nella stessa componente fortemente connessa $C(u)$, che per forza di cose parte da $T_u$ ed esce da questo sottoalbero per arrivare a $\alpha$. Sia $(v, w)$ il primo arco uscente da $T_u$ (ossia $v \in T_u$ e $w \notin T_u$). Questo arco non può essere ovviamente in avanti perché altrimenti avremmo $w \in T_u$, dunque o è all'indietro o è di attraversamento. Vediamo entrambi i casi:

- $(v, w)$ è all'indietro
  In questo caso quindi $w$ è un antenato di $v$, e siccome $w \notin T_u$, questo è anche antenato di $u$, e perciò questo è stato sicuramente visitato prima di $u$, dunque $t(w) < t(u)$. Essendo però $u$ discendente di $w$, sappiamo che la chiamata ricorsiva di $w$ è ancora aperta, e dunque $w$ è ancora in `Open`.
- $(v, w)$ è di attraversamento
  In questo caso, per la Proposizione 1.16, troviamo che $t(w) < t(u)$ (altrimenti $w$ avrebbe fatto parte del sottoalbero $T_u$). Poiché $(v, w)$ è un arco facente parte del cammino tra $u$ e $\alpha$, dato che $\alpha$ e $u$ fanno parte della stessa componente fortemente connessa troviamo che $w$ fa parte anche lui della componente $C(u)$. Dato che la chiamata ricorsiva su $\alpha$ ancora deve finire, essendo antenato di $u$, troviamo che $w$ è ancora in `Open`, dato che un nodo esce dallo stack quando finisce la chiamata ricorsiva della c-radice della sua componente.

Possiamo quindi concludere che $t(w) < t(u)$ e che $w \in$ `Open`. ■

2° $u$ non è una c-radice $\impliedby$ esiste un arco $(v, w)$ tale che $t(w) < t(u)$ e $w \in$ `Open`
Dato che $w \in$ `Open`, sappiamo che $w$ è stato visitato ma la sua componente ancora non è stata definita. Dunque, trovandoci noi ora nella chiamata ricorsiva di $u$, sappiamo sicuramente che la c-radice della componente $C(w)$, che chiamiamo $z$, è un antenato di $u$, dunque esiste sicuramente un cammino da $z$ a $u$, e dato che esiste un cammino da $w$ a $z$, esiste un cammino anche da $w$ a $u$. Ma esiste anche un cammino da $u$ a $w$, di cui fa parte proprio l'arco $(v, w)$. Dunque $u$ e $w$ fanno parte della stessa componente fortemente connessa, e dato che $w$ è visitato prima di $u$, sicuramente $u$ non può essere la c-radice di questa componente. ■

Grazie a questa proposizione, riusciamo a trovare un nuovo algoritmo `DFS_SCC` che, per capire se un nodo è una c-radice, sfrutta una logica molto simile a quella usata per l'algoritmo

`CercaPonti_ric` visto per trovare tutti i ponti in un grafo. Vediamolo prima e cerchiamo di capirlo:

```
DFS_SCC(G): // G grafo diretto
    n = numero di nodi di G
    Open = stack nuovo
    CC = [0] * n
    t = 0
    comp = 0
    for u in V(G):
        if CC[u] == 0:
            DFS_SCC_ric(G,u,CC,Open,t,comp)
    return CC
```

dove `DFS_SCC_ric` è questa funzione ricorsiva:

```
DFS_SCC_ric(G,u,CC,Open,t,comp):
    t++
    CC[u] = -t
    Open.push(u)
    back = t
    for v con arco (u,v):
        if CC[v] == 0: // Nodo mai visitato
            b = DFS_SCC_ric(G,v,CC,Open,t,comp)
            back = min(back,b)
        else if CC[v] < 0: // Nodo visitato ma di cui non conosciamo ancora la
            componente fortemente connessa
            back = min(back, -CC[v])
    if back == -CC[u]:
        comp++
        do:
            w = Open.pop()
            CC[w] = comp
        while w != u
    return back
```

Come nell'algoritmo visto per i ponti utilizziamo qui la variabile `back`. In questa inseriamo il valore di visita minore che troviamo percorrendo tutti gli archi con nodi di cui ancora non conosciamo la componente fortemente connessa nel sottoalbero $T_u$. In particolare, se troviamo un nodo che non è stato ancora visitato rifacciamo una chiamata ricorsiva e controlliamo quanto indietro andiamo seguendo il suo sottoalbero, mentre se troviamo un nodo che è già stato visitato ma di cui non conosciamo la componente, controlliamo solo se il suo tempo di visita è minore di `back` e nel caso lo aggiorniamo. Se in questa ricerca troviamo anche solo un arco

che porta a un nodo $w$ tale che $t(w) < t(u)$, ossia $back < -CC[u]$ (infatti in questo algoritmo $CC[u] = -t[u]$ quando non conosciamo ancora la componente di $u$, e diventa uguale a $comp$ solo quando capiamo in che componente si trova), allora siamo sicuri che $u$ non è una c-radice per la Proposizione 1.18.
Restituiamo quindi un array finale `CC` dove all'indice i-esimo corrisponde il numero della componente del nodo i-esimo. Ad esempio, per il grafo in Figura 1.5 avremmo in output l'array `[3,3,2,2,2,1,3]` (infatti, la prima componente trovata è {6}, la seconda {5,4,3} e la terza {7,2,1}). Il costo totale di questo algoritmo è semplicemente $\mathbf{O}(n + m)$, essendo di base un DFS con solo alcune modifiche che però apportano un costo di O(1) (come già visto per l'algoritmo dei ponti). Questo algoritmo che abbiamo studiato prende anche il nome di **algoritmo di Tarjan**.

## 1.7 BFS

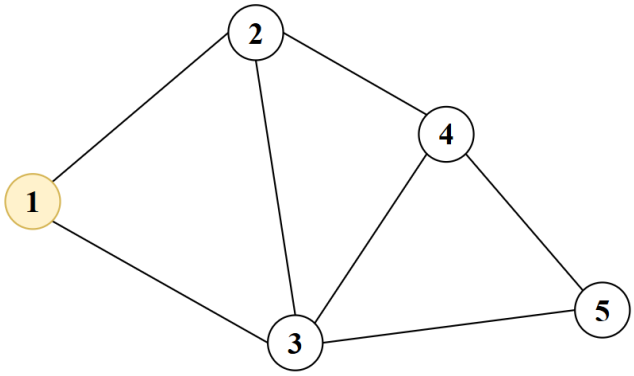
Torniamo finalmente a parlare ora di BFS. Questo algoritmo, che ricordiamo non lavora in profondità come il DFS ma piuttosto per livelli, controllando tutti i vicini possibili di un nodo prima di andare avanti, ci è utile per risolvere un problema molto importante, ossia trovare la distanza tra due nodi $x$ e $y$ in un grafo $G$ non diretto e connesso. Ricordiamo che la distanza tra due nodi è il numero di archi di cui è composto il cammino minimo che collega i due nodi.

Per risolvere questo problema usiamo un algoritmo che, preso un nodo $x$, trova la distanza da $x$ a tutti i nodi del grafo. Questo sfrutta un ragionamento molto semplice: a partire da $x$, sappiamo già che $dist(x, x) = 0$ (ossia che la distanza tra $x$ e $x$ è ovviamente pari a 0), e sappiamo che, $\forall z$ adiacente a $x$, $dist(x, z) = 1$. A questo punto, per ogni vicino $z'$ di $z$ che non abbiamo ancora raggiunto troviamo che $dist(x, z') = dist(x, z) + dist(z, z') = 2$. Continuando così possiamo calcolare la distanza da $x$ a un qualsiasi nodo in $G$. Vediamo come si traduce questo in un vero e proprio algoritmo:

```
BFS(G,x): // G grafo non diretto e connesso
    n = numero di nodi di G
    dist = [-1] * n # Array con le distanze di ogni nodo da x
    dist[x] = 0
    Q = queue vuota
    Q.enqueue(x)
    while Q != 0:
        y = Q.dequeue()
        for z adiacente a y:
            if dist[z] == -1: // z ancora mai visitato
                dist[z] = dist[y] + 1
                Q.enqueue(z)
    return dist
```

Sfruttiamo quindi in questo caso una coda dove inseriamo tutti i nodi non appena vengono visitati per la prima volta. Per ogni nodo $y$ poi controlliamo i vicini e, per tutti quelli ancora da visitare, calcoliamo la distanza con la stessa formula che abbiamo visto prima, ossia distanza $dist(x, y) + 1$, e alla fine restituiamo l'array con tutte le distanze.
In totale il costo sarà dato da questa formula: $\sum_{y \in V(G)} O(1 + \deg(y))$. Infatti, $O(1)$ perché ogni nodo viene inserito nella queue al più una volta, e $O(\deg(y))$ dato che per ogni nodo eseguiamo un for che legge ogni suo vicino. Il costo totale sarà quindi $\sum_{y \in V(G)} O(1) + \sum_{y \in V(G)} O(\deg(y)) = O(n) + O(2m) = \mathbf{O(n + m)}$.

Vediamo con un esempio come funziona questo algoritmo:




In questo caso usiamo 1 come nodo di partenza, e calcoliamo tutte le distanze da lui.

1° Chiamata su 1: `Q = [1], dist = [0,-1,-1,-1,-1]`
2° Fine chiamata su 1 / Chiamata su 2: `Q = [2,3], dist = [0,1,1,-1,-1]`
3° Fine chiamata su 2 / Chiamata su 3: `Q = [3,4], dist = [0,1,1,2,-1]`
4° Fine chiamata su 3 / Chiamata su 4: `Q = [4,5], dist = [0,1,1,2,2]`
5° Fine chiamata su 4 / Chiamata su 5: `Q = [5], dist = [0,1,1,2,2]`
6° Fine chiamata su 5: `Q = [], dist = [0,1,1,2,2]`
7° Ritorniamo `dist = [0,1,1,2,2]`

**Osservazione 1.6**

$x$ e $y$ si trovano su due **componenti diverse** $\implies$ $\boldsymbol{dist(x, y) = \infty}$.

Questo caso non viene gestito dal nostro algoritmo perché partiamo con l'ipotesi che $G$ sia non diretto e connesso.

**Proposizione 1.19**

Preso un grafo non diretto $G$ e due vertici $x, y \in V(G)$, $dist(x, y) = k$ con $k \geq 1 \implies \exists$ un vicino $z$ di $y$ tale che $\boldsymbol{dist(x, z) = k - 1}$.

*Dimostrazione*

Sia $P$ il cammino tra $x$ e $y$ di lunghezza $k$. Prendiamo ora l'arco finale di $P$ $zy$. Il cammino tra $x$ e $z$ è ovviamente lungo $k - 1$, dunque sicuramente $dist(x, z) \leq k - 1$ (perché la distanza è il cammino minimo, che può essere anche più corto di $k - 1$ ma sicuramente non più lungo). Ma non solo, possiamo con certezza dire che $dist(x, z) = k - 1$: infatti, se così non fosse, vorrebbe dire che avremmo un cammino tra $x$ e $z$ di lunghezza $j < k - 1$, e quindi avremmo un cammino tra $x$ e $y$ di lunghezza $j + 1 < k$, ma questo è impossibile dato che $dist(x, y) = k$. Possiamo quindi concludere che $dist(x, z) = k - 1$. ■

Grazie a questa proposizione, ci è possibile dimostrare che l'array `dist` nell'algoritmo che abbiamo appena visto contenga effettivamente le distanze tra $x$ e tutti gli altri nodi. Infatti, prima abbiamo scritto la BFS sfruttando l'intuizione che, presa $dist(x, y) = k$, ogni vicino $z$ di $y$ non ancora raggiunto nel momento in cui $y$ viene estratto dalla coda si trovi a distanza $k + 1$. Dimostriamolo ora per induzione.

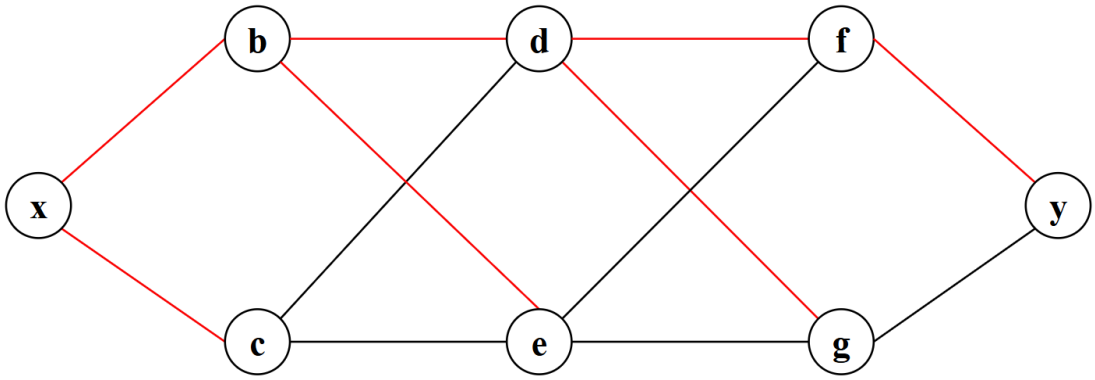
- Caso base: $dist(x, x)$
  Poniamo esplicitamente `dist[x] = 0`, dunque troviamo che `dist[x]`$= dist(x, x)$. ■
- Ipotesi induttiva
  Assumiamo che, $\forall z$ tali che $dist(x, z) = k - 1$, si abbia anche $dist[z] = k - 1$
- Passo induttivo
  Preso $y$ tale che $dist(x, y) = k$, sappiamo per la Proposizione 1.19 che esiste un vicino $z$ di $y$ tale che $dist(x, z) = k - 1$, e per ipotesi induttiva `dist[z] = k-1`. Ora, la coda estrae i nodi in ordine di `dist` non decrescente (ogni nodo viene inserito con `dist` pari a quello del nodo appena estratto +1), quindi $z$ viene estratto prima di qualsiasi nodo con `dist` $\geq k$. Dunque $y$ viene scoperto per la prima volta da un nodo $w$ con `dist[w]` $\leq k - 1$, ossia `dist[y]` $\leq k$. D'altra parte `dist[y]` non può essere minore di $k$, perché ogni valore di `dist` è la lunghezza di un cammino reale da $x$, e il più corto è lungo $k$. Troviamo quindi proprio che `dist[y] = k`. ■

Dunque ora possiamo dire con certezza che la BFS appena studiata trovi tutte le distanze tra un nodo e tutti gli altri nodi.

### 1.7.1 Trovare quanti cammini minimi esistono tra due nodi

Vogliamo ora invece trovare un modo per trovare **quanti** cammini minimi esistono tra due nodi $x$ e $y$, ossia quanti cammini tra $x$ e $y$ sono lunghi $dist(x, y)$. Per capire come fare, guardiamo

questo esempio:




In rosso è stata già evidenziata una BFS completa. Con questa, riusciamo a trovare solo un cammino minimo da $x$ a $y$, ma è evidente che questo non sia l'unico. Per capire quanti cammini minimi veramente esistono tra $x$ e $y$ ci conviene partire dai nodi vicini a $x$, ossia $b$ e $c$. Ovviamente per arrivare a loro abbiamo un solo cammino minimo, e questo anche la BFS ce lo dice. Passiamo ora ai vicini di $b$ e $c$: $d$ ha 2 cammini minimi possibili, e anche $e$ ne ha 2. Ne hanno proprio 2 perché esistono ben due vicini a loro che distano da $x$ esattamente quanto loro $-1$, ossia $b$ e $c$, e quindi entrambi questi nodi ci permettono di arrivare a $d$ o $e$ con un cammino di lunghezza minima. Questo stesso ragionamento può essere fatto per $f$ e $g$: dato che hanno 2 vicini che distano da $x$ quanto loro $-1$, il numero di cammini minimi tra $x$ e loro è pari alla somma dei cammini minimi tra $x$ e $d$ e tra $x$ e $e$, ossia 4. Seguendo questo stesso ragionamento, troviamo che il numero di cammini minimi tra $x$ e $y$ è proprio $4 + 4 = 8$. Vediamo come si traduce questo in codice:

```
BFS_cammini_minimi(G,x):
    n = numero di nodi in G
    dist = [-1] * n
    count = [0] * n // Array con il numero di cammini minimi
    Q = queue vuota
    Q.enqueue(x)
    dist[x] = 0
    count[x] = 1
    while Q != 0:
        y = Q.dequeue()
        for z adiacente a y:
            if dist[z] == -1: # z ancora non visitato
                dist[z] = dist[y] + 1
                count[z] = count[y] # Tutti i cammini minimi a y portano anche a z
                Q.enqueue(z)
            else: # z già visitato
                if dist[z] == dist[y] + 1: # Se è un vicino con distanza uguale ma -1
```

```
                    count[z] += count[y] # Aggiungiamo altri cammini
    return count
```

Il costo rimane ovviamente $\mathbf{O(n+m)}$, essendo un BFS leggermente modificato.

### 1.7.2 Distanza tra due sottoinsiemi

Vediamo ora un altro problema. Presi due sottoinsiemi $X$ e $Y$ di un grafo $G$ non diretto e connesso, cerchiamo di trovare la distanza tra questi due sottoinsiemi, definita in questo modo: $\boldsymbol{dist(X,Y) = \min_{x\in X \wedge y \in Y} dist(x,y)}$, ossia la distanza minima tra un elemento di $X$ e uno di $Y$.

Un'idea che può venire subito in mente è ovviamente fare tanti BFS quanti sono gli elementi di $X$ e trovare la distanza minima tra uno di questi elementi e un nodo in $Y$. Questo però ha costo $O(n(n+m))$, dato che richiede di effettuare tanti BFS quanti sono gli elementi di $X$, che potrebbero essere in un numero nell'ordine di $n$. Cerchiamo invece un algoritmo con un costo pari a $O(n+m)$. Abbiamo due possibilità:

- Opzione A: Creiamo un nuovo nodo $x'$ collegato a tutti i nodi di $X$, calcoliamo la distanza minima tra $x'$ e $Y$ e diminuiamo la distanza di 1, così da calcolare la distanza minima tra $X$ e $Y$. Creare un nodo costa $O(n)$ e calcolare la distanza minima costa $O(n+m)$, dunque il costo totale è $O(n+m)$ come richiesto.
- Opzione B: Eseguiamo un BFS totale su $X$ inserendo tutti i suoi nodi nella queue fin da subito, ponendo tutti i loro valori `dist` pari a 0. Anche questa ha costo $O(n+m)$ perché utilizza un semplice BFS.

Useremo l'opzione B perché non richiede l'aggiunta di alcun nodo. L'algoritmo è il seguente:

```
BFS_Sot(G,X,Y):
    n = numero di nodi di G
    dist = [-1] * n
    Q = queue vuota
    for x in X: // Inseriamo tutti gli elementi nella coda
        Q.enqueue(x)
        dist[x] = 0
    while Q != 0: // BFS normale
        y = Q.dequeue()
        for z adiacente a y:
            if dist[z] == -1: // z ancora mai visitato
                dist[z] = dist[y] + 1
                Q.enqueue(z)
    return dist
```

Il costo di questo algoritmo rimane ovviamente $\mathbf{O(n + m)}$ essendo praticamente un normale BFS.

### 1.7.3 Alberi BFS

**Proposizione 1.20**

Preso un grafo non diretto $G$ e tre nodi $x, y, z \in V(G)$, se `dist` è l'array con le distanze da $x$ a tutti gli altri nodi calcolato tramite un BFS, $y$ adiacente a $z \implies$ **`|dist[z] - dist[y]|`** $\leq 1$.

*Dimostrazione*

Analizziamo tre casi separati:

- Se `dist[z]` = `dist[y]` allora `dist[z]` − `dist[y]` = 0.
- Se `dist[z]` < `dist[y]` allora, essendo questi due nodi adiacenti, `dist[y]` ≤ `dist[z]` +1 (basta arrivare a $z$ e poi usare l'arco $zy$), quindi, essendo le distanze numeri interi, troviamo che `dist[y]` = `dist[z]` +1, ossia `dist[z]` − `dist[y]` = −1
- Se `dist[z]` > `dist[y]` allora, essendo questi due nodi adiacenti, `dist[z]` ≤ `dist[y]` +1 (basta arrivare a $y$ e poi usare l'arco $yz$), quindi, essendo le distanze numeri interi, troviamo che `dist[z]` = `dist[y]` +1, ossia `dist[z]` − `dist[y]` = 1

Concludiamo quindi che | **`dist[z] - dist[y]`** | ≤ 1. ■

**Proposizione 1.21 – Corollario**

Preso un grafo $G$ non diretto e un albero $T$ derivante da un BFS su tale grafo, tutti gli archi fuori da $T$ collegano due nodi con distanze uguali o che differiscono di 1.

Questo corollario viene proprio dalla proposizione sopra: infatti, se due nodi sono adiacenti in un grafo, abbiamo appena dimostrato che la loro differenza sarà sempre minore o uguale a 1, dunque tutti gli archi che collegano due nodi in G collegano due nodi con distanze uguali o che differiscono di 1. Questo, se poteva sembrare scontato per gli archi in $T$, di certo non lo era per gli archi fuori da $T$.

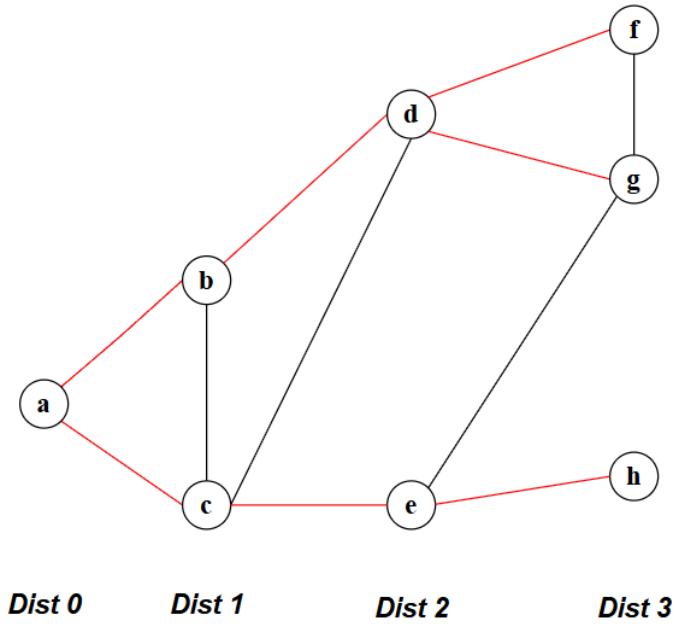



**Figura 1.6:** Esempio albero BFS

In questo esempio si vede benissimo il corollario appena enunciato. Il BFS è rappresentato dagli archi rossi, che compongono quindi l'albero $T$, mentre gli archi neri sono tutti gli altri archi fuori da $T$ ma in $G$. Notiamo proprio come tutti questi archi neri colleghino sempre due nodi con distanze uguali o che differiscono di 1.

**Proposizione 1.22**

Preso un grafo $G$ non diretto e connesso, un nodo $x \in V(G)$, un albero BFS $T_1$ e un altro albero DFS $T_2$, entrambi con radice $x$, $T_1 = T_2 \iff G$ è un albero.

*Dimostrazione*

- $T_1 = T_2 \implies G$ è un albero
  Questa prima parte della proposizione possiamo dimostrarla dimostrando la negazione, ossia che $G$ non è un albero $\implies T_1 \neq T_2$ (infatti, $A \implies B$ equivale a dire $\neg A \impliedby \neg B$).
  Se $G$ non è un albero, allora, essendo questo connesso, sappiamo che esiste un ciclo al suo interno. L'esistenza di un ciclo implica l'esistenza di un arco all'indietro per quanto riguarda il DFS, per la Proposizione 1.8. Chiamiamo questo arco $yz$. Essendo questo un arco all'indietro, sappiamo che uno dei due nodi è antenato dell'altro e che tra i due nodi c'è almeno un nodo di distacco (altrimenti esisterebbero due archi $yz$, quello del DFS normale e quello all'indietro). Dunque questo arco collega due nodi con distanze da $x$ che

differiscono di almeno 2, cosa impossibile in un albero BFS. Concludiamo che $T_2$ non è un albero BFS, ossia $T_1 \neq T_2$. ■

- $T_1 = T_2 \impliedby G$ è un albero
  Questa seconda parte della proposizione è banale: se $G$ è un albero allora, essendo anche $T_1$ e $T_2$ degli alberi con all'interno tutti i nodi di $G$ (essendo alberi rispettivamente BFS e DFS), $G = T_1 = T_2$. ■